Tensor Networks Meet Neural Networks: A Survey and Future Perspectives

Maolin Wang*, Yu Pan*, Zenglin Xu**, *Senior Member, IEEE*, Guangxi Li, Xiangli Yang, Danilo Mandic, *Fellow, IEEE*, and Andrzej Cichocki, *Fellow, IEEE*

arXiv:2302.09019v3 [cs.LG] 17 Mar 2025

Abstract—Tensor networks (TNs) and neural networks (NNs) are two fundamental data modeling approaches. TNs were introduced to solve the curse of dimensionality in large-scale tensors by converting an exponential number of dimensions to polynomial complexity. As a result, they have attracted significant attention in the fields of quantum physics and machine learning. Meanwhile, NNs have displayed exceptional performance in various applications, e.g., computer vision, natural language processing, and robotics research. Interestingly, although these two types of networks originate from different observations, they are inherently linked through the typical multilinearity structure underlying both TNs and NNs, thereby motivating a significant number of developments regarding combinations of TNs and NNs. In this paper, we refer to these combinations as tensorial neural networks (TNNs) and present an introduction to TNNs from both data processing and model architecture perspectives. From the data perspective, we explore the capabilities of TNNs in multi-source fusion, multimodal pooling, data compression, multi-task training, and quantum data processing. From the model perspective, we examine TNNs' integration with various architectures, including Convolutional Neural Networks, Recurrent Neural Networks, Graph Neural Networks, Transformers, Large Language Models, and Quantum Neural Networks. Furthermore, this survey also explores methods for improving TNNs, examines flexible toolboxes for implementing TNNs, and documents TNN development while highlighting potential future directions. To the best of our knowledge, this is the first comprehensive survey that bridges the connections among NNs and TNs. We provide a curated list of TNNs at https://github.com/tnbar/awesome-tensorial-neural-networks.

Index Terms—Tensor Networks, Deep Neural Networks, Network Compression, Information Fusion, Quantum Circuit Simulation

1 Introduction

Tensors are higher-order arrays that represent multiway interactions among multiple modal sources. In contrast, vectors (i.e., first-order tensors) and matrices (i.e., second-order tensors) are accessed in only one or two modes, respectively. As a common data type, tensors have been widely observed in several scenarios [1], [2], [3], [4]. For instance, functional magnetic resonance imaging (fMRI) samples are inherently fourth-order tensors that are composed of three-dimensional voxels that change over time [5], [6], [7], [8], [9]. In quantum physics, variational wave functions used to study many-body quantum systems are also high-order tensors [10], [11], [12]. For spatiotemporal traffic analysis, road flow/speed information, which is collected from multiple roads over several weeks, can also be structured as a third-order tensor (road segment×day×time of day) [13]. However, for higher-order tensors, when the number of modes increases, the total number of elements in the tensors grows exponentially, which is prohibitive for storing and processing tensors, which is also recognized as the "curse of dimensionality" [14]. Tensor networks are common and effective methods to mitigate this problem.

Tensor Networks (TNs). TNs [10], [14], [15] are generally countable collections of small-scale tensors that are interconnected by tensor contractions. These small-scale tensors are referred to as "components", "blocks", "factors", or "cores". Very-large-scale tensors can be approximately represented in extremely compressed and distributed formats through TNs. Thus, it is feasible to implement distributed storage and efficient processing for high-order tensors that could not be dealt with before. By using TN methods, the curse of dimensionality can be alleviated or completely overcome [14]. Commonly used TN formats include CANDECOMP/PARAFAC (CP) [16], [17], [18], Tucker decomposition [19], [20], Block-term Tucker (BTT) decomposition [21], Matrix Product State (MPS)/Tensor Train (TT) decomposition [22], [23], [24], Matrix Product Operators (MPO)/matrix Tensor Train (mTT) decomposition [22], [23], [24], Tensor Ring (TR) decomposition [25], Tree TN/Hierarchical Tucker (HT) decomposition [26], Projected Entangled Pair State (PEPS)/Tensor Grid decomposition [10], [27], Multiscale Entanglement Renormalization [28], etc. For the purpose of understanding the interconnected structures of TNs, a TN diagram was developed as a straightforward graphical diagram (which is discussed in Section 2.2). A TN can provide a theoretical and computational framework for the analysis of some computationally prohibitive tasks. For example, based on the low-rank structures of TNs, Pan *et al.* [29] were able to solve the quantum random circuit sampling problem in 15 hours using 512 graphics processing units (GPUs); this problem was previously believed to require over 10,000 years on the most powerful classic electronic supercomputer and effectively challenge the quantum

* *Equal Contribution*
** *Corresponding Author*

- *M. Wang is with the City University of Hong Kong, HKSAR, China. E-mail: morin.w98@gmail.com*
- *Y. Pan is with the Harbin Institute of Technology Shenzhen, Shenzhen, China. E-mail: iperryuu@gmail.com*
- *Z. Xu is with the Fudan University at the Shanghai Academy of AI for Science, Shanghai, China. E-mail: zenglin@gmail.com*
- *G. Li is with the Quantum Science Center of Guangdong-Hong Kong-Macao Greater Bay Area, Shenzhen, China. E-mail: gxli2017@gmail.com*
- *X. Yang is with the Information Engineering University in Zhengzhou, China. E-mail: xlyang@std.uestc.edu.cn*
- *D. Mandic is with the Department of Electrical and Electronic Engineering, Imperial College London, UK, E-mail: d.mandic@imperial.ac.uk*
- *A. Cichocki is with the Systems Research Institute, Polish Academy of Sciences, Newelska 6, 01-447 Warsaw, Poland, and also with Artificial Intelligence Project, Riken, 103-0027 Tokyo, Japan, E-mail: a.cichocki@riken.jp*

TABLE 1: An overview of TNNs from both Data and Model Perspectives. This table presents different categories including both data processing approaches and model architectures.

Category	Subcategory	Detailed Models/Techniques	Section
Data Processing	Multi-source Fusion	TFL [33], LMF [34], PTP [35], HPFN [35], Deep Polynomial NN [36]	3.1
	Multimodal Pooling	MCB [37], MLB [38], MUTAN [39], CTI [40]	3.2
	Data Compression	BNTD [41], TensorCodec [42], [43], NeuKron [44], Light-IT and Light-IT++ [42], TT-PC [45], TTHRESH [46] M^2DMTF [47], Lee *et al.* [48], Lamba *et al.* [49], FLEST [50]	3.3
	Multi-task Training	TTMT [51], TMT [51], PEPS-like TN [52], MTCN [53], Zhang *et al.* [54], GTTN [55], CTNN [56], M2TD [57], MRI [58], FTN [59], MULTIPAR [60] Liu *et al.* [61], WISDOM [62], MMER-TD [63]	3.4
	Quantum Data	Quantum State Mapping [10], [64], Word Quantum Embedding [65], [66], [67], [68]	3.5
Model Architecture	CNNs	CP-CNN [69], [70], [71], [72], [73], Tucker-CNN [74], [75], BTT-CNN [76], TT-CNN [77], [78], [79], TR-CNN [80], T-Net [81], TR-Compress [82], CPD-EPC [72], CP-HOConv [83]	4.1
	RNNs	TT-RNN [84], TR-RNN [85], BTT-RNN [76], [86], TT-GRU [87], HT-RNN [88], HT-TT [89], Conv-TT-LSTM [90], TC-Layer [91], MPS-NLP [92], CP-RNN [74], Tucker-RNN [74]	4.2
	Transformers	MPO-Transformer [93], Hypoformer [94], Tucker-Bert [95], MMT [96], TCTN [97],T6 [98] Tuformer [99], Tensorial Causal Learning [100]	4.3
	GNNs	TGNN [101], TGCN [102], Nimble GNN [103], RTGNN [104], THNNs [105], DSTGNN [106]	4.4
	QNNs	MPS Models [122], Born Machine [123], ConvAC [124], [125], TSLM [126], ANTN [127], ADTN [128], TTLM [129], TFNs [130]	4.5
	LLMs	Model Compression: TensorGPT [107], CompactifAI [108], FASTER-LMs [109], TTM [110], TQCompressor [111] Parameter-Efficient Fine-tuning: TT-LoRA [112], SuperLoRA [113], Quantum-PEFT [114], LoRA-PT [115], FLoRA [116], LoTR [117], Quantum-inspired-PEFT [118], QuanTA [119], FacT [120], DoTA [121]	4.6

supremacy of Google's quantum computer called "Sycamore". Other applications include brain analysis [30], dimensionality reduction [31], subspace learning [32], etc.

Neural Networks (NNs). NNs are powerful learning structures that enable machines to acquire knowledge from observed data [171], [172]. Deep Neural Networks (DNNs) [173], [174], which stack multiple layers of neural processing units, have revolutionized artificial intelligence by demonstrating unprecedented capabilities in capturing complex patterns and representations from hierarchical structures. The DNN family encompasses various architectural paradigms, including restricted Boltzmann machines (RBMs) [175] for unsupervised learning, convolutional neural networks (CNNs) [174], [176] for spatial pattern recognition, recurrent neural networks (RNNs) [177], [178] for sequential data processing, and Transformers [179], [180] for attention-based learning. DNNs have achieved remarkable breakthroughs across diverse domains, particularly in computer vision [181] and natural language processing [182]. In computer vision, the evolution of CNN architectures marks significant milestones in image classification on the ImageNet dataset [183], from AlexNet [184] to VGGNet [185], GoogLeNet [186], and ResNet [187], each introducing novel architectural innovations. A groundbreaking achievement in structural biology came with AlphaFold2 [188], [189], which revolutionized protein structure prediction by reducing the time required from years to days and successfully predicting the structures of nearly all known proteins with remarkable atomic precision. The field of natural language processing has witnessed a paradigm shift with the emergence of large language models (LLMs). Models such as ChatGPT [190], Qwen [191], Llama [192], Claude 3 [193], DeepSeek [194], [195], and ChatGLM [196], built upon Transformer architectures, have demonstrated capabilities matching or exceeding human performance across diverse professional and academic tasks. The impact of deep learning continues to expand across numerous scientific and practical domains. These include advancing speech recognition systems [197], enhancing DNA mutation detection methods [198], revolutionizing structural biology research [199], accelerating drug discovery processes [200], improving food security measures [201], and demonstrating the versatility and transformative potential of neural network approaches.

Tensor Networks Meet Neural Networks. Tensor Networks (TNs) and Neural Networks (NNs), while stemming from distinct scientific foundations, have each demonstrated unique capabilities across diverse domains, as documented in earlier discussions. Despite their different origins, recent research highlights a deep connection through their multilinear mathematical structures, thus challenging the once presumed orthogonality between them [14], [202]. TNs are particularly appreciated for their efficient architectures and prowess in handling heterogeneous data sources. In contrast,

TABLE 2: An overview of TNN utility. This table presents different utility aspects of TNNs, including training strategies and various toolboxes for implementation and processing.

Category	Subcategory	Detailed Models/Techniques	Section
Training Strategy	Stable Training	Mixed Precision [131], Yu Initialization [132], MANGO [133]	5.1
	Rank Selection	PSTRN [134], TR-RL [135], CP-Bayes [136], PARS [137], TT-Bayes [138], Adaptive TR [139], TT-ADMM [140], BMF [141], Gusak *et al.* [142], Solgi *et al.* [143]	5.2
	Hardware Speedup	TIE [144], LTNN [145], TT-Engine [146], Fast CP-CNN [147], ETTE [148], Huang *et al.* [149], T2s-tensor [150], Tensaurus [151], Xie *et al.* [152], Liang *et al.* [153], Fawzi *et al.* [154]	5.3
Toolboxes	Basic Tensor Operations	Tensorly [155], TensorTools [156], Tensor Toolbox [157], HOTTBOX [158], TenDeC++ [159], OSTD [160], TensorD [161], TT-Toolbox [162], Tntorch [163], TorchMPS [122], ITensor [164], T3F [165], TensorNetwork [166], Scikit-TT [167]	6.1
	Deep Model Implementations	Tensorly-Torch [155], TedNet [74]	6.2
	Quantum Tensor Simulations	Yao [168], TensorNetwork [166], lambeq [169], ITensor [164], TeD-Q [170]	6.3

NNs are acclaimed for their broad utility in many fields [10], [15]. Notably, emerging studies explore potential mappings between TNs and NNs, suggesting profound synergistic relationships [203], [204]. We argue that integrating TNs with NNs can markedly enhance model performance and sustainability in AI from both data and model perspectives. From a computational sustainability standpoint, TNNs offer improved data efficiency through their structured representations, requiring fewer training samples and computational resources. Their parameter-efficient nature aligns well with the growing emphasis on sustainable AI development, potentially reducing the environmental impact of model training and deployment. Moreover, the theoretical foundations of TNs provide a mathematical framework for understanding and improving neural network architectures, potentially leading to more efficient and interpretable AI systems. We argue that integrating TNs with NNs can markedly enhance model performance and sustainability in AI from both data and model perspectives:

(1) Effective Data Representation: Accurate modeling of higher-order interactions from multi-source data is critical in advancing performance and promoting responsible AI practices [10]. Conventional NNs, which typically process inputs as flat vectors, often fall short in effectively capturing complex data interrelations [39]. Direct modeling of these interactions risks the 'curse of dimensionality,' leading to prohibitively high training or processing costs. Integrating TNs within NN frameworks presents a powerful solution, exploiting TNs' capability to manage multi-entry data efficiently. This approach facilitates robust processing in multimodal, multiview, and multitask scenarios, enhancing both performance and accountability [33], [35], [205]. For example, the Multimodal Tucker Fusion (MUTAN) technique leverages a Tucker decomposition to foster high-level interactions between textual and visual data in VQA tasks, achieving leading results while fostering the design of power-efficient, ethically oriented AI systems with a low-rank, efficient parameter structure [39], [206]. Additionally, the TensorCodec approach [43], employing Tensor-Train Decomposition, effectively compresses data, supporting sustainable AI efforts and enhancing our ability to interpret and utilize complex datasets.

(2) Compact Model Structures: NNs have achieved significant success across various applications. However, their high computational demands, especially for high-dimensional data and the associated curse of dimensionality, remain a substantial challenge [86]. TNs offer a sustainable alternative by harnessing their intrinsic lightweight and multilinear properties to address these issues effectively [74], [75], [80], [85], [86]. By decomposing neural network weight tensors into smaller, manageable components, TNs transform the computational complexity from an exponential to a linear scale [70], [71], [80], [85], [86]. A prime example is the TR-LSTM model, which employs TN techniques to decompose weight tensors in action recognition tasks, reducing parameters by approximately 34,000 times while enhancing performance beyond traditional LSTM models [85]. Such innovations are crucial for the advancement of Sustainable AI, promoting the development of algorithms that are both effective and environmentally considerate.

We refer to the this family of approaches that connect TNs with NNs as **tensorial neural networks (TNNs)**. Although this combination holds significant promise for sustainable AI by offering efficient parameter compression and structured representations, TNNs also present new training challenges that require careful consideration. These challenges include numerical stability issues during optimization, particularly for high-order tensor operations and decompositions, complex hyperparameter selection especially for determining optimal tensor ranks and network architectures, and hardware acceleration requirements to efficiently handle tensor contractions and parallel computations. Therefore, it is necessary to redesign traditional neural network training techniques to address these TNN-specific challenges. While existing surveys on tensor networks have primarily focused on introducing fundamental TN concepts or their applications in specific domains such as image processing, signal processing, or quantum computing, they often treat neural networks and tensor networks as separate methodologies. To the best of our knowledge, this is the first comprehensive survey to systematically bridge the connections between NNs and TNs, providing a unified view of their integration, challenges, and solutions.

An overview of both their data processing capabilities and model architectures of TNNs is shown in Table 1. From the data processing perspective, TNNs demonstrate versatility across multiple domains including: multi-source fusion for integrating heterogeneous data sources, multimodal pooling for efficient feature

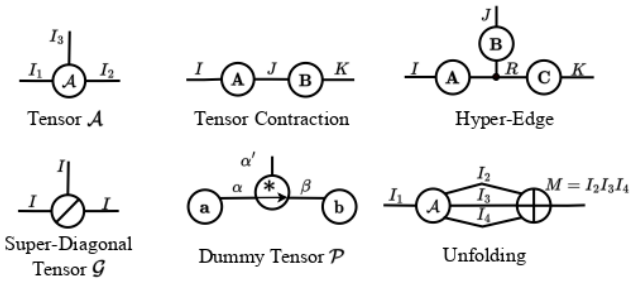


Fig. 1: Basic symbols for TN diagrams. For more details about TNs, refer to [10] and [14].

TABLE 3: Tensor notations

Symbol	Explanation
a	scalar
$\mathbf{a}$	vector
$\mathbf{A}$	matrix
$\boldsymbol{\mathcal{A}}$	tensor
A	dimensionality
$\circledast$	convolution operation
$\circ$	outer product operation
$< \cdot, \cdot >$	inner product of two tensors
$\|\cdot\rangle$	quantum state bra vector (unit column complex vector)
$\langle\cdot\|$	quantum state ket vector (unit row complex vector)
$\langle\cdot\|\cdot\rangle$	inner product of two quantum state vectors

combination, data compression for reducing storage requirements while preserving information fidelity, multi-task training for simultaneous learning of related objectives, and quantum data processing for handling quantum state representations. From the model architecture perspective, TNNs have been successfully integrated into various deep learning frameworks including CNNs, RNNs, Transformers, GNNs, Large Language Models (LLMs), and Quantum Neural Networks (QNNs), each offering unique advantages in their respective application domains. Table 2 provides a comprehensive overview of TNN practical utilities, focusing on training strategies and implementation aspects. The training strategies encompass stable training techniques for numerical stability, rank selection methods for optimal tensor decomposition, and hardware acceleration approaches for efficient deployment. The toolbox ecosystem includes libraries for basic tensor operations, deep model implementations, and quantum tensor simulations, facilitating both research and practical applications of TNNs.

The remaining sections of this survey are organized as follows. Section 2 provides the fundamentals of tensor notations, tensor diagrams, and TN formats. Section 4 discusses the use of TNs for building compact TNNs. Section 3 explores efficient information fusion processes using TNNs. Section 5 explains some training and implementation techniques for TNNs. Section 6 introduces general and powerful toolboxes that can be used to process TNNs.

2 Tensor Basis

2.1 Tensor Notations

A tensor [207], [208], also known as a multiway array, can be viewed as a higher-order extension of a vector (i.e., a first-order tensor) or a matrix (i.e., a second-order tensor). Like the rows and columns in a matrix, an Nth-order tensor $\boldsymbol{\mathcal{X}} \in \mathbb{R}^{I_1 \times I_2 \cdots \times I_N}$ has N modes (i.e., ways, orders, or indices) whose lengths (i.e., dimensions) are represented by I_1 to I_N, respectively. As shown in Table 3, lowercase letters denote scalars, e.g., a, boldface lowercase letters denote vectors, e.g., $\mathbf{a}$, boldface capital letters denote matrices, e.g., $\mathbf{A}$ and boldface Euler script letters denote higher-order tensors, e.g., $\boldsymbol{\mathcal{A}}$. In this paper, we define a "tensor" in a broad sense that includes scalars, vectors, and matrices.

2.2 Tensor Diagrams

In this subsection, we introduce TN diagrams and their corresponding mathematical operations. TN diagrams were first developed by Roger Penrose [209] in the early 1970s and are now commonly used to describe quantum algorithms [10], [11] and machine learning algorithms [15], [64], [71]. Within these diagrams, tensors are denoted graphically by nodes with edges [23], which enables intuitive presentation and convenient representation of complex tensors. As both the data and weights in the deep learning field are tensors, tensor diagrams are also promising for use as general network analysis tools in this area. An overview of the basic symbols of tensors is shown in Fig. 1.

2.2.1 Tensor Nodes

A tensor is denoted as a node with edges, as illustrated in Fig. 1. The number of edges denotes the modes of a tensor, and a value on an edge represents the dimension of the corresponding mode. For example, a one-edge node denotes a vector $\mathbf{a} \in \mathbb{R}^I$, a two-edge node denotes a matrix $\mathbf{A} \in \mathbb{R}^{I \times J}$ and a three-edge node denotes a tensor $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$.

2.2.2 Tensor Contraction

Tensor contraction refers to the operation whereby two tensors are contracted into one tensor along their associated pairs of indices. As a result, the corresponding connected edges disappear while the dangling edges persist. Tensor contraction can be formulated as a tensor product

$$\boldsymbol{\mathcal{C}} = \boldsymbol{\mathcal{A}} \times^{1,2,\ldots N}_{M+1,M+2,\ldots,M+N} \boldsymbol{\mathcal{B}} \tag{1}$$

with the elements of $\boldsymbol{\mathcal{C}}$ are computed via

$$\boldsymbol{\mathcal{C}}_{p_1,\ldots,p_{K+M}} = \sum_{i_1,i_2,\ldots i_N} \boldsymbol{\mathcal{A}}_{i_1,i_2,\ldots i_N,*} \quad \boldsymbol{\mathcal{B}}_{*,i_1,i_2,\ldots i_N}, \tag{2}$$

where $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_1 \times \ldots I_N \times P_1 \times \ldots P_K}$, $\boldsymbol{\mathcal{B}} \in \mathbb{R}^{P_{K+1} \times \ldots P_{K+M} \times I_1 \times \ldots I_N}$, and $\boldsymbol{\mathcal{C}} \in \mathbb{R}^{P_1 \times \ldots P_K \times P_{K+1} \ldots P_{K+M}}$. Fig. 1 also shows a diagram of the matrix multiplication operation, which is the most classic tensor contraction situation, given by:

$$\mathbf{C} = \mathbf{AB} = \mathbf{A} \times^1_2 \mathbf{B}. \tag{3}$$

Tensor contractions among multiple tensors (e.g., TNs) can be computed by sequentially performing tensor contractions between each pair of tensors. It is worth mentioning that the contracting sequence must be determined to achieve better calculation efficiency [152].

2.2.3 Dummy Tensor

Recently, a newly designed dummy tensor was proposed by Hayashi *et al.* to represent convolution operations [71]. As depicted in Fig. 1, a node with the star and arrow symbols denotes a dummy tensor. This operation is formulated as

$$\mathbf{y}_{j'} = \sum_{j=0}^{\alpha-1} \sum_{k=0}^{\beta-1} \boldsymbol{\mathcal{P}}_{j,j',k} \mathbf{a}_j \mathbf{b}_k, \tag{4}$$

where $\mathbf{a} \in \mathbb{R}^{\alpha}$ denotes a vector that will be processed by a convolutional weight $\mathbf{b} \in \mathbb{R}^{\beta}$, and $\mathbf{y} \in \mathbb{R}^{\alpha'}$ is an output. The

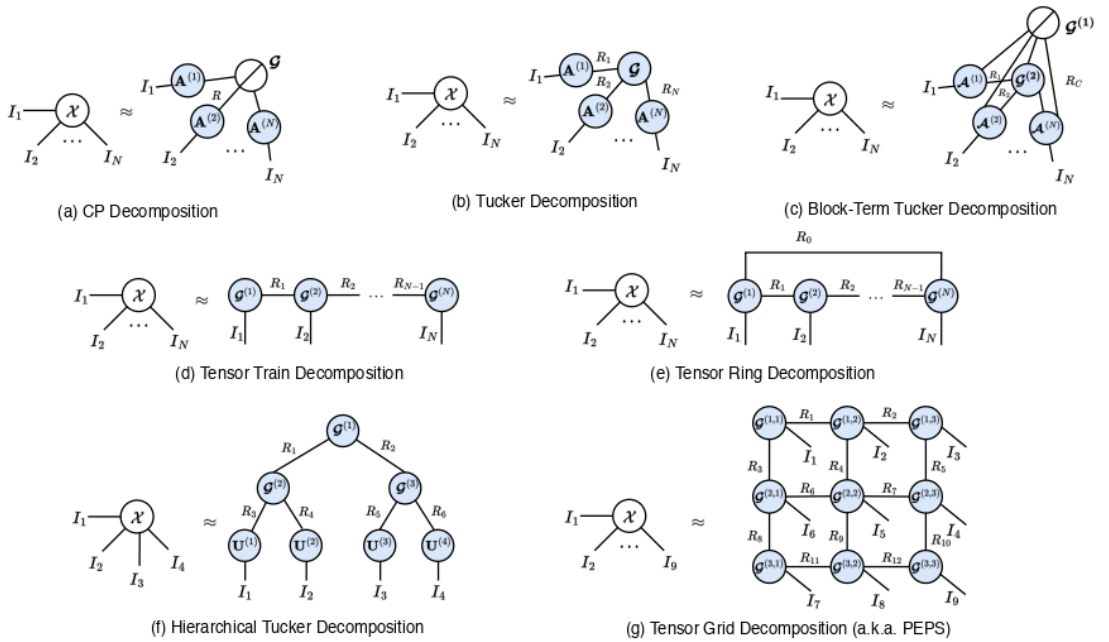


Fig. 2: TN diagrams of some popular TN decompositions. (a) The CP format decomposes a tensor $\mathcal{X}$ into a sum of several rank-1 tensors $\boldsymbol{a}_{:,r}^{(1)} \circ \boldsymbol{a}_{:,r}^{(2)} \circ \cdots \circ \boldsymbol{a}_{:,r}^{(N)}$. (b) Tucker decomposition decomposes a tensor $\mathcal{X}$ into a core tensor $\mathcal{G}$ multiplied by a matrix $\boldsymbol{A}^{(n)}$ along the nth mode. (c) Block term decomposition decomposes a tensor $\mathcal{X}$ into a sum of several Tucker decompositions (on the right) with low Tucker ranks. (d) TT decomposition decomposes a tensor $\mathcal{X}$ into a linear multiplication of a set of 3rd-order core tensors $\mathcal{G}^{(2)} \cdots \mathcal{G}^{(N-1)}$ and two matrices $\mathcal{G}^{(1)}$, $\mathcal{G}^{(N)}$. (e) TR decomposition decomposes a tensor $\mathcal{X}$ into a set of 3rd-order core tensors and contracts them into a ring structure. (f) HT decomposition represents a tensor $\mathcal{X}$ as a tree-like diagram. For more basic knowledge about TNs, we refer to [10] and [14]. (g) Tensor Grid Decomposition (a.k.a. PEPS) represents a high-dimensional tensor as a two-dimensional grid of interconnected lower-rank tensors, where each node connects to its neighbors to efficiently capture spatial correlations in systems with local interactions.

symbol $\mathcal{P} \in \{0,1\}^{\alpha \times \alpha' \times \beta}$ denotes a binary tensor with elements defined as $\mathcal{P}_{j,j',k} = 1$ if $j = sj' + k - p$ and 0 otherwise, where s and p represent the stride and padding size, respectively. Thus, $\mathcal{P}$ can be applied to any two tensors to form a convolutional relationship.

2.2.4 Hyperedge

Fig. 1 illustrates the hyperedge that was also introduced by Hayashi *et al.* [71]. An example of a hyperedge with a size of R can be formulated as

$$\mathcal{Y}_{ijk} = \sum_{r=1}^{R} \mathbf{A}_{ir}\mathbf{B}_{jr}\mathbf{C}_{kr}, \tag{5}$$

where $\mathbf{A} \in \mathbb{R}^{I \times R}$, $\mathbf{B} \in \mathbb{R}^{J \times R}$ and $\mathbf{C} \in \mathbb{R}^{K \times R}$ are three matrices. $\mathcal{Y} \in \mathbb{R}^{I \times J \times K}$ denotes the results of applying a hyperedge on $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$. A hyperedge node represents a specialized tensor where diagonal elements are set to 1, serving as a crucial component in tensor network diagrams. This tensor functions as an addition operator, enabling the combination of multiple substructures (such as the matrices illustrated in Fig. 1) into a unified representation. The significance of hyperedge nodes was demonstrated by Hayashi *et al.* [71] in their groundbreaking work on tensorial CNNs (TCNNs). They proved that any TCNN architecture can be fully represented using a tensor network diagram through the strategic placement of dummy tensors and hyperedges.

2.2.5 Super-diagonal Tensor

A super-diagonal tensor is a tensor whose entries outside the main diagonal are all 0 and whose dimensionality is the same as its order. An Nth-order super-diagonal tensor $\mathcal{G} \in \mathbb{R}^{\overbrace{I \times I \times \cdots \times I}^{N}}$ is a tensor with elements defined as $\mathcal{G}_{i_1,i_2,\cdots i_N} \in \mathbb{R}$ if $i_1 = i_2, \cdots = i_N$ and 0 otherwise. As shown in Figure 1, a super-diagonal tensor is designated by a node with a skew line in TN diagrams. The identity tensor $\mathcal{I}$ is a special super-diagonal tensor with all entries on the main diagonal equal to one. A hyperedge can be regarded as performing a tensor contraction operation with an identity tensor.

2.2.6 Tensor Unfolding

Tensor unfolding is an operation that virtually flattens a tensor into a high-dimensional but low-order tensor. Matricization is a special case of tensor unfolding. To be more specific, given an Nth-order tensor $\mathcal{A} \in \mathbb{R}^{I_1 \times I_2 \cdots \times I_N}$, its mode-$n$ unfolding process yields a matrix $\boldsymbol{A}_{(n)} \in \mathbb{R}^{I_n \times I_1 I_2 \cdots I_{n-1} I_{n+1} \ldots I_N}$. Such an operation can also be regarded as performing tensor contraction with a specifically designed tensor. A fourth-order tensor unfolding diagram is illustrated in Fig. 1.

2.3 Tensor Decomposition Formats

The commonly used terminology "tensor decomposition" (TD) is equivalent to "tensor network" to some extent. While TD was employed primarily in signal processing fields [210], [211], TNs were originally utilized largely in the physics and quantum circuit fields [10], [209]. Traditional TD models, such as CP [16], [17], [18] and Tucker decomposition [19], [20], can be viewed as basic kinds of TNs. In the realm of signal processing, several powerful TNs architectures for quantum analysis have also been introduced. For instance, the MPS decomposition [212] was defined as a

TT decomposition [22] and had tremendous success in several applications [15]. After years of collaboration and progress across different research fields, there is no significant distinction between these two terminologies. Therefore, TD and TNs are treated in a unified way in this paper. We briefly introduce some basic TDs by employing TN diagrams.

2.3.1 CANDECOMP/PARAFAC

The CP decomposition [16], [17], [18] factorizes a higher-order tensor into a sum of several rank-1 tensor components. For instance, given an Nth-order tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \cdots I_N}$, each of its elements in the CP format can be formulated as

$$\mathcal{X}_{i_1,i_2,\ldots,i_N} \approx \sum_{r=1}^{R} \mathcal{G}_r \prod_{n=1}^{N} \mathbf{A}^{(n)}_{i_n,r}, \tag{6}$$

where R denotes the CP rank (defined as the smallest possible number of rank-1 tensors [210]), $\mathcal{G}$ denotes the N-th-order super-diagonal tensor, and $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R}$ denotes a series of factor matrices. The TN diagram for CP is illustrated in Fig. 2 (a).

When calculating a CP format, the first issue that arises is how to determine the number of rank-1 tensor components, i.e., the CP rank R. Actually, this is an NP-hard problem [213]. Hence, in practice, a numerical value is usually assumed in advance (i.e., as a hyperparameter), to fit various CP-based models [210]. After that, the diagonal core tensor $\mathcal{G}$ and the factor matrices $\mathbf{A}^{(n)}$ can be directly solved by employing algorithmic iteration, which usually involves the alternating least-squares (ALS) method that was originally proposed in [16], [17].

2.3.2 Tucker Decomposition

Tucker decomposition [19], [20] factorizes a higher-order tensor into a core tensor multiplied by a corresponding factor matrix along each mode. To be more specific, given an Nth-order tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \cdots I_N}$, the Tucker decomposition can be formulated in an elementwise manner as

$$\mathcal{X}_{i_1,i_2,\ldots,i_N} \approx \sum_{r_1,\ldots,r_N=1}^{R_1,\ldots,R_N} \mathcal{G}_{r_1,r_2,\ldots,r_N} \prod_{n=1}^{N} \mathbf{A}^{(n)}_{i_n,r_n}, \tag{7}$$

where $\{R_1, R_2, \ldots, R_N\}$ denotes a series of Tucker ranks, $\mathcal{G} \in \mathbb{R}^{R_1 \times R_2 \ldots R_N}$ denotes the dense core tensor and $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R_n}$ denotes a factor matrix. The TN diagram for Tucker decomposition is illustrated in Fig. 2 (b). Please note that compared with the CP rank, $R_1, R_2, \ldots, R_N$ can take different numerical values.

Tucker decomposition is commonly used and can be degraded to CP by setting the core tensor $\mathcal{G}$ as a super diagonal tensor. In addition, the original Tucker decomposition lacks constraints on its factors, leading to the nonuniqueness of its decomposition results, which is typically undesirable for practical applications due to the lack of explainability. Consequently, orthogonality constraints are always imposed on the component matrices, yielding the well-known higher-order singular value decomposition (HOSVD) algorithm [214].

2.3.3 BTT Decomposition

The CP and Tucker decompositions both decompose a tensor into a core tensor multiplied by a matrix along each mode, while CP imposes an additional super diagonal constraint on the core tensor for the sake of simplifying the structural information of the core tensor. A more generalized decomposition method called the BTT decomposition [21] has been proposed as a tradeoff between the CP and Tucker methods, by imposing a block diagonal constraint on Tucker's core tensor. The TN diagram for the BTT decomposition is illustrated in Fig. 2 (c).

The BTT decomposition aims to decompose a tensor into a sum of several Tucker decompositions with low Tucker ranks. Specifically, the BTT decomposition of a 4th-order tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times I_3 \times I_4}$ can be represented by 6 nodes with special contractions. Here, $\mathcal{G}^{(1)}$ denotes the 5-th-super-diagonal tensor, $\mathcal{G}^{(2)} \in \mathbb{R}^{R_C \times R_T \times R_T \times R_T \times R_T}$ denotes the R_C core tensors of the Tucker decompositions, and each $\mathcal{A}^{(n)} \in \mathbb{R}^{R_C \times I_n \times R_T}$ denotes the R_C corresponding factor matrices of the Tucker decompositions. Moreover, each element of $\mathcal{X}$ is computed as

$$\begin{aligned}\mathcal{X}_{i_1,i_2,i_3,i_4} \approx \sum_{r_C=1}^{R_C} \mathcal{G}^{(1)}_{r_C} \sum_{r_1,r_2,r_3,r_4=1}^{R_T,R_T,R_T,R_T} \\ \mathcal{G}^{(2)}_{r_C,r_1,r_2,r_3,r_4} \mathcal{A}^{(1)}_{r_C,i_1,r_1} \mathcal{A}^{(2)}_{r_C,i_2,r_2} \mathcal{A}^{(3)}_{r_C,i_3,r_3} \mathcal{A}^{(4)}_{r_C,i_4,r_4},\end{aligned} \tag{8}$$

where R_T denotes the Tucker rank (which means that the Tucker rank equals $\{R_T, R_T, R_T, R_T\}$) and R_C represents the CP rank. Together, they are called BT ranks.

The advantages of BTT decomposition mainly depend on its compatibility with the benefits of the both CP and Tucker methods. The reason for this is that when the Tucker rank is equal to 1, BTT decomposition degenerates to CP; when the CP rank equals 1, it degenerates to Tucker decomposition.

2.3.4 Tensor Train Decomposition

The TT decomposition [22], [23], also known as Matrix Product State (MPS) decomposition in quantum physics [212], [215], is a fundamental tensor network approach that originates from quantum many-body physics. This decomposition method factorizes a higher-order tensor into a sequence of third-order core tensors connected through matrix multiplications. For an Nth-order tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \ldots I_N}$, the TT decomposition can be expressed elementwise as

$$\begin{aligned}\mathcal{X}_{i_1,i_2,\ldots,i_N} \approx \sum_{r_1,r_2,\ldots,r_{N-1}=1}^{R_1,R_2,\ldots,R_{N-1}} \\ \mathcal{G}^{(1)}_{1,i_1,r_1} \mathcal{G}^{(2)}_{r_1,i_2,r_2} \mathcal{G}^{(3)}_{r_2,i_3,r_3} \cdots \mathcal{G}^{(N)}_{r_{N-1},i_N,1},\end{aligned} \tag{9}$$

where $\{R_1, R_2, \ldots, R_{N-1}\}$ are the TT ranks, $\mathcal{G}^{(n)} \in \mathbb{R}^{R_{n-1} \times I_n \times R_n}$ represents a third-order core tensor, and $R_0 = R_N = 1$, making $\mathcal{G}^{(1)}$ and $\mathcal{G}^{(N)}$ effectively matrices. The network structure of TT decomposition is visualized in Fig. 2 (d).

One of the key advantages of TT decomposition is its computational tractability, as it can be efficiently computed through recursive applications of Singular Value Decomposition (SVD). Specifically, the decomposition process sequentially unfolds the tensor into matrices, applies SVD to obtain core tensors, and continues this process along each dimension, making it numerically stable and algorithmically efficient. The computational complexity scales linearly with the tensor order, making it particularly attractive for high-dimensional problems. Being the most straightforward among tensor network models due to its linear structure and well-understood mathematical properties, TT decomposition has found widespread applications in both theoretical development and practical implementations of tensor networks [11]. Its simplicity and efficiency have made it a cornerstone for parameter compression in deep learning, quantum state simulation, high-dimensional function approximation, and numerical linear algebra.

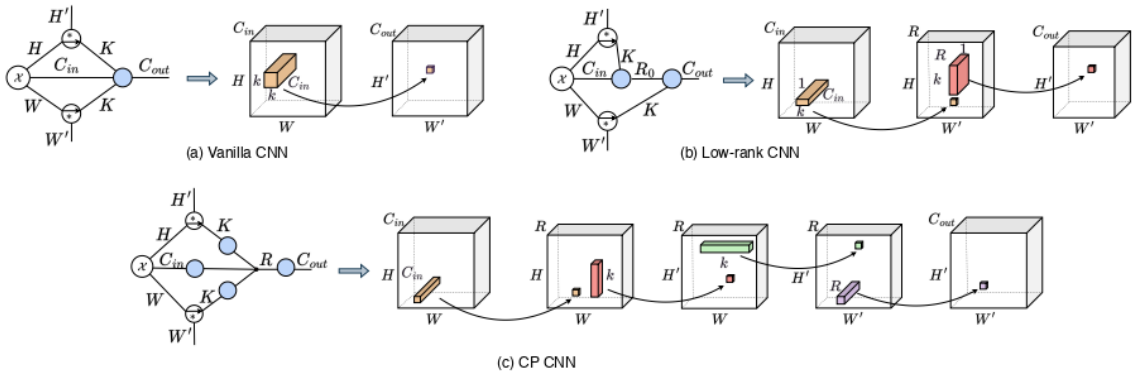


Fig. 3: Correspondence between TN diagrams and convolutional procedures. In each subfigure, the left part is a TN diagram, and the right part is the associated commonly used feature representation.

While Eq. (9) and Fig. 2 (d) demonstrate the MPS format, some research works [84], [94], [216] have extended TT decomposition to utilize the Matrix Product Operator (MPO) [217] format. For a $2N$-order tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times J_1 \times I_2 \times J_2 \ldots I_N \times J_N}$, the MPO decomposition takes the form

$$\mathcal{X}_{i_1,j_1,i_2,j_2\ldots,i_N,j_N} \approx \sum_{r_1,r_2,\ldots,r_{N-1}=1}^{R_1,R_2,\ldots,R_{N-1}} \mathcal{G}^{(1)}_{1,i_1,j_1,r_1} \mathcal{G}^{(2)}_{r_1,i_2,j_2,r_2} \mathcal{G}^{(3)}_{r_2,i_3,j_3,r_3} \cdots \mathcal{G}^{(N)}_{r_{N-1},i_N,j_N,1}, \tag{10}$$

where $\{R_1, R_2, \ldots, R_{N-1}\}$ denote the ranks controlling the complexity and expressiveness of the decomposition, $\mathcal{G}^{(n)} \in \mathbb{R}^{R_{n-1} \times I_n \times I_n \times R_n}$ represents a fourth-order core tensor that captures the local correlations and interactions between adjacent tensor modes, and the boundary conditions $R_0 = R_N = 1$ are imposed to ensure proper tensor contraction, which effectively reduces $\mathcal{G}^{(1)}$ and $\mathcal{G}^{(N)}$ to third-order core tensors acting as the terminal components of the decomposition chain.

2.3.5 Tensor Ring Decomposition

The TT benefits from fast convergence; however, it suffers from the effects of its two endpoints, which hinder the representation ability and flexibility of TT-based models. Thus, to release the power of linear architectures, researchers have linked its endpoints to produce a ring format named a tensor ring [25], [218], [219], [220]. The TR decomposition of a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \ldots I_N}$ can be formulated as

$$\mathcal{X}_{i_1,i_2,\ldots,i_N} \approx \sum_{r_0,r_1,\ldots,r_{N-1}}^{R_0,R_1,\ldots,R_{N-1}} \mathcal{G}^{(1)}_{r_0,i_1,r_1} \mathcal{G}^{(2)}_{r_1,i_2,r_2} \mathcal{G}^{(3)}_{r_2,i_3,r_3} \cdots \mathcal{G}^{(N)}_{r_{N-1},i_N,r_0}, \tag{11}$$

where $\{R_0, R_1, \ldots, R_N\}$ denote the TR ranks, each node $\mathcal{G}^{(n)} \in \mathbb{R}^{R_{n-1} \times I_n \times R_n}$ is a 3rd-order tensor and $R_0 = R_N$. Compared with TT decomposition, it is not necessary for TR decomposition to follow a strict order when multiplying its nodes. The TN diagram for TR decomposition is illustrated in Fig. 2 (e).

2.3.6 Hierarchical Tucker Decomposition

The HT decomposition [26] possesses a tree-like structure. In general, it is feasible to connect a tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times \cdots \times I_N}$ to a binary tree with a root node associated with $S_{set} = \{1, 2, \cdots, N\}$ and $\mathcal{X} = \mathcal{U}_{S_{set}}$ as the root frame. The notation $S_{set1}, S_{set2} \subseteq S_{set}$ is defined as the set that is associated with the left child node $\mathcal{U}_{S_{set1}}$ and right child node $\mathcal{U}^{S_{set2}}$, while $\mathcal{U}^{S_{set1}} \in \mathbb{R}^{R_1 \times I_{\min(S_{set1})} \times \cdots \times I_{\max(S_{set1})}}$ can also be recursively decomposed into its left child node $\mathcal{U}^{D_{set1}}$ and right child node $\mathcal{U}^{D_{set1}}$. The first three steps are as

$$\mathcal{U}^{S_{set}} \approx \mathcal{G}^{s} \times_1^2 \mathcal{U}^{S_{set1}} \times_1^2 \mathcal{U}^{S_{set2}}, \tag{12}$$

$$\mathcal{U}^{S_{set1}} \approx \mathcal{G}^{s1} \times_1^2 \mathcal{U}^{D_{set1}} \times_1^2 \mathcal{U}^{D_{set2}}, \tag{13}$$

$$\mathcal{U}^{S_{set2}} \approx \mathcal{G}^{s2} \times_1^2 \mathcal{U}^{D_{set3}} \times_1^2 \mathcal{U}^{D_{set4}}, \tag{14}$$

where $\mathcal{G}^{s} \in \mathbb{R}^{R_1 \times R_2}$, $\mathcal{G}^{s1} \in \mathbb{R}^{R_1 \times R_3 \times R_4}$ and $\mathcal{G}^{s2} \in \mathbb{R}^{R_2 \times R_5 \times R_6}$. This procedure can be performed recursively to obtain a tree-like structure. The TN diagram for HT decomposition is illustrated in Fig. 2 (f).

2.3.7 PEPSs Decomposition

TN structures with different typologies and higher-dimensional connections can also be considered. One such structure is the PEPS decomposition [10], [27], [221], also known as tensor grid decomposition [222], which is a high-dimensional TN that generalizes a TT. PEPS decomposition provides a natural structure that can capture more high-dimensional information, while PEPS cores can be characterized as $\mathcal{G}^{(m,n)} \in \mathbb{R}^{I_{mn} \times R_{l_{mn}} \times R_{r_{mn}} \times R_{u_{mn}} \times R_{d_{mn}}}$.

The mathematical formulation of PEPS decomposition [52] can be expressed as

$$\mathcal{X}_{i_1,i_2,\ldots,i_{MN}} = \sum_{h^{(R)},h^{(C)}} \sum_{m,n} \mathcal{G}^{(m,n)}_{i_{mn};h^{(R)}_{l_{mn}},h^{(R)}_{r_{mn}},h^{(C)}_{u_{mn}},h^{(C)}_{d_{mn}}}, \tag{15}$$

where the indices follow a structured pattern defined by

$$\begin{cases} l_{mn} = (n-2)M + m, \\ r_{mn} = (n-1)M + m, \\ u_{mn} = (m-2)N + n, \\ d_{mn} = (m-1)N + n, \\ R^{(R)}_i = 1, \quad \text{while} \quad i < 0 \quad or \quad i > M(N-1), \\ R^{(C)}_i = 1, \quad \text{while} \quad i < 0 \quad or \quad i > N(M-1). \end{cases} \tag{16}$$

Here, M and N represent the number of rows and columns in the tensor core arrangement, respectively. The ranks $h^{(R)}_i$ and $h^{(C)}_j$ characterize the bond dimensions along the row and column directions, controlling the amount of quantum entanglement or classical correlation that can be captured across these directions. The topological structure of PEPS decomposition is visualized in Fig. 2 (g). A distinguishing feature of PEPS decomposition is its

polynomial correlation decay with respect to the separation distance, which stands in contrast to the exponential correlation decay exhibited by MPS decomposition. This fundamental difference in correlation behavior demonstrates the superior representational capacity of PEPS [10], enabling more effective modeling of long-range interactions and complex correlations between different tensor modes in the network structure.

3 SUSTAINABLE AI THROUGH TNN IN DATA ASPECT: EFFECTIVE DATA REPRESENTATION

In real-world data analysis, information often comes from multiple sources, such as vision, sound, and text in video data [206], [223]. A prime example is the Visual Question Answering (VQA) task, where the key challenge lies in effectively modeling interactions between textual and visual information. Processing such diverse data sources uniformly is impractical, necessitating specialized architectures with multiple input channels to handle multimodal sources - an approach known as information fusion. While traditional methods like feature-level fusion [224] and decision-level fusion [225] were popular in early stages, these linear approaches failed to effectively model intramodality dynamics. Tensor Neural Networks (TNNs) have emerged as a solution, leveraging their natural multilinear properties to model intramodality dynamics and process higher-order data. TNNs provide effective frameworks for tensor operations, making them naturally suited for expressing and generalizing information fusion modules commonly found in deep learning, such as attention mechanisms and vector concatenation [226]. As a result, numerous studies have adopted TNNs to capture higher-order interactions among data or parameters. In the following sections, we will explore various TNN-based approaches for data representation and processing. First, we examine advanced data compression techniques that leverage tensor network architectures to achieve significant parameter reduction while preserving critical information structures. We then investigate novel tensor fusion layers (Section 3.1) designed to facilitate deep feature interactions and transformations across modalities, followed by sophisticated multimodal data pooling mechanisms (Section 3.2) that effectively integrate information across different data types.

3.1 Multi-source Data fusion

Multimodal sentiment analysis is a task containing three communicative modalities, i.e., the textual modality, visual modality, and acoustic modality [33]. Addressing multimodal sentiment analysis, Zadeh *et al.* [33] proposed novel TNNs with deep information fusion layers named tensor fusion layers (TFLs), which can easily learn intramodality dynamics and intermodality dynamics and are able to aggregate multimodal interactions, thereby efficiently fusing the three communicative modalities. Specifically, a TFL first takes embedded feature vectors $\mathbf{z_t}$, $\mathbf{z_v}$ and $\mathbf{z_a}$ derived by embedding networks rather than the original three data types. Then, the TFL concatenates a scalar 1 with each embedded feature vector as

$$\mathbf{z}_t^{'} = \begin{bmatrix} \mathbf{z}_t \\ 1 \end{bmatrix} \mathbf{z}_v^{'} = \begin{bmatrix} \mathbf{z}_v \\ 1 \end{bmatrix} \mathbf{z}_a^{'} = \begin{bmatrix} \mathbf{z}_a \\ 1 \end{bmatrix}. \tag{17}$$

Then, as shown in Fig. 4, the TFL obtains a feature tensor $\mathcal{Z}$ by calculating the outer product among the three concatenated vectors

$$\mathcal{Z} = \mathbf{z}_t^{'} \circ \mathbf{z}_v^{'} \circ \mathbf{z}_a^{'} = \begin{bmatrix} \mathbf{z}_t \\ 1 \end{bmatrix} \circ \begin{bmatrix} \mathbf{z}_v \\ 1 \end{bmatrix} \circ \begin{bmatrix} \mathbf{z}_a \\ 1 \end{bmatrix}. \tag{18}$$

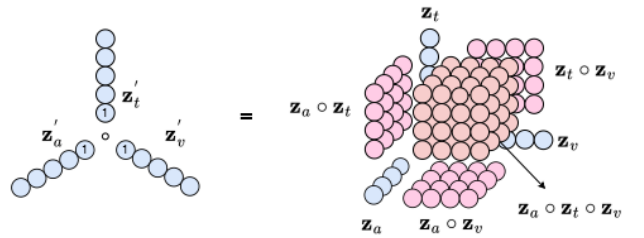


Fig. 4: Illustration of the tensor fusion process in Eq. (18). Different from a TN diagram, each circle corresponds to a value.

Finally, the TFL processes the feature tensor $\mathcal{Z}$ to obtain a prediction $\mathbf{y}$ via a two-layer fully connected NN. Compared to direct concatenation-based fusion, which only considers unimodal interactions [33], the TFL benefits from capturing both unimodal interactions and multimodal interactions.

Despite its success, the TFL suffers from an exponential increase in its computational complexity and number of parameters when the number of modalities increases. For example, in a multimodal sentiment analysis case [33], the feature tensor $\mathcal{Z} \in \mathbb{R}^{129\times33\times33}$ and the hidden vector $\mathbf{h} \in \mathbb{R}^{128}$ can result in $17,981,568$ parameters to be optimized. To address these excessive parameters, low-rank multimodal fusion (LMF) [34] adopts a special BTT layer to overcome the massive computational cost and overfitting risks of the TFL. For a general situation with n modalities, the feature tensor $\mathcal{Z} = \circ_{m=1}^{M} z_m^{'}$ can be processed. The hidden vector $\mathbf{h}$ can be computed as follows

$$\mathbf{h} = ReLU\left(\boldsymbol{he}(\mathbf{W}_1 z_1^{'}, \mathbf{W}_2 z_2^{'}, \ldots, \mathbf{W}_M z_M^{'}, I) + \mathbf{b}\right),$$

where $\mathbf{W}_i \in \mathbb{R}^{d_i \times d_h}$ is the weight matrix and $I \in \mathbb{R}^{d_h \times d_h}$ is an identity matrix. The LMF reduces the computational complexity of the TFL from $O\left(\prod_{m=1}^{M} d_m\right)$ to $O\left(d_h \times \sum_{m=1}^{M} d_m\right)$.

Although LMF and the TFL achieve better fusion results than other methods, they restrict the order of interactions, causing higher-order interactions to lack information. A PTP [35] block has been proposed to tackle this problem. The whole procedure and TN diagram of PTP are shown in Fig. 5 and Fig. 6, respectively.

The PTP first merges all feature vectors $\{\mathbf{z}_m\}_{m=1}^{M}$ into a long feature vector

$$\mathbf{z}_{12\cdots M}^{\top} = \left[1, \mathbf{z}_1^{\top}, \mathbf{z}_2^{\top}, \ldots, \mathbf{z}_M^{\top}\right]. \tag{19}$$

The polynomial feature tensor of degree P is represented as

$$\mathcal{Z}^P = \mathbf{z}_{12\ldots M} \circ \mathbf{z}_{12\ldots M} \circ \cdots \circ \mathbf{z}_{12\cdots M}. \tag{20}$$

The PTP [35] then adopts a tensorial layer (e.g., a CP layer) to process the polynomial feature tensor $\mathcal{Z}^P$. The CP layer is represented as

$$\mathbf{h} = \boldsymbol{he}(\mathbf{W}_1 \mathbf{z}_{12\ldots M}, \ldots \mathbf{W}_P \mathbf{z}_{12\ldots M}, \boldsymbol{\Lambda}) \tag{21}$$

where $\mathbf{W}_i \in \mathbb{R}^{d_i \times d_h}$ is the weight matrix and $\boldsymbol{\Lambda} \in \mathbb{R}^{d_h \times d_h}$ is a learnable diagonal matrix. The structure of PTP is also equivalent to that of a deep polynomial NN [36], whereby PTP models all nonlinear high-order interactions. For multimodal time series data, one approach uses a "window" to characterize local correlations and stack the PTP blocks into multiple layers. Such a model is called a hierarchical polynomial fusion network (HPFN) [35]. The HPFN can recursively process local temporal-modality patterns to achieve a better information fusion effect.

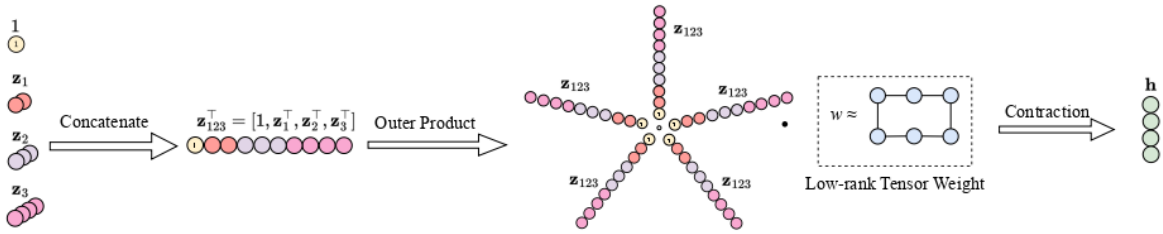


Fig. 5: Illustration of polynomial tensor pooling (PTP) [35]. PTP first concatenates all feature vectors $\mathbf{z}_1, \mathbf{z}_2, \mathbf{z}_3$ into a longer feature vector $\mathbf{z}_{123}^\top = [1, \mathbf{z}_1^\top, \mathbf{z}_2^\top, \mathbf{z}_3^\top]$, it then derives a polynomial feature tensor by repeatedly performing outer product operations on the feature vector $\mathbf{z}_{123}$, and finally adopts a tensorial layer (e.g., a TR layer) to merge the polynomial feature tensor into a vector $\mathbf{h}$.

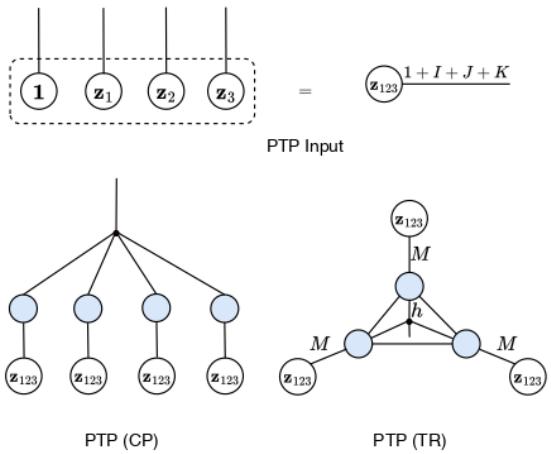


Fig. 6: TN diagrams of PTP. The CP and TR structures can be adopted in such a strategy.

The structure of a single-layer PTP block is similar to that of a shallow convolutional arithmetic circuit (ConvAC) network [67] (see Section 3.5 and 4.5). The only difference between ConvAC and PTP is that the standard ConvAC network processes quantum location features, whereas PTP processes the temporal-modality patterns and polynomial concatenated multimodal features. The HPFN is nearly equivalent to a deeper ConvAC network, and its great expressive power might be implied by their connection. The recursive relationships in deep polynomial NNs have also been found and implemented so that polynomial inputs can be efficiently computed via a hierarchical NN [35]. Chrysos *et al.* [36] also discovered similar results.

3.2 Multimodal Data Pooling

Another group of information fusion methods originated from VQA tasks [206]. In VQA tasks, the most important aspect is to parameterize bilinear the interactions between visual and textual representations. To address this aspect, some tensor fusion methods have been discovered in this area. Multimodal compact bilinear pooling (MCB) [37] is a well-known fusion method for VQA tasks and can be regarded as a special Tucker decomposition-based NN, which tries to optimize the simple bilinear fusion operation

$$\mathbf{z} = \mathbf{W}[\mathbf{v} \circ \mathbf{q}], \tag{22}$$

where $\mathbf{v}$ and $\mathbf{q}$ are input vectors with different modalities and $\mathbf{W}$ is a learnable weight matrix. Moreover, MCB optimizes the computational cost of the outer product operation based on the property of the count sketch projection function.

Multimodal low-rank bilinear pooling (MLB) [38] adopts a CP layer in a data fusion step that can be formulated as

$$\mathbf{z} = \mathbf{1}^T \left(\mathbf{W}_v \mathbf{v} \circ \mathbf{W}_q \mathbf{q}\right), \tag{23}$$

where $\mathbf{W}_q$ and $\mathbf{W}_v$ are the prepossessing weight matrices for the inputs $\mathbf{q}$ and $\mathbf{v}$, respectively and $\mathbf{1}$ is a vector in which all values are 1. The structure of the MLB method is a special case of LMF (see Sec. 3.1). MLB fusion methods can also be regarded as simple product pooling when the number of modalities is equal to two.

The MUTAN [39] is a generalization of MCB and MLB, which adopts a Tucker layer to learn the bilinear interactions between visual and textual features as

$$\begin{aligned} \mathbf{z} &= \left(\left(\mathcal{T}_c \times_1^1 \left(\mathbf{q}^\top \mathbf{W}_q\right)\right) \times_2^1 \left(\mathbf{v}^\top \mathbf{W}_v\right)\right) \times_3^1 \mathbf{W}_o, \\ \mathbf{z} &= \left(\mathcal{T}_c \times_1^1 \tilde{\mathbf{q}}\right) \times_2^1 \tilde{\mathbf{v}}, \end{aligned} \tag{24}$$

where $\tilde{\mathbf{q}} = \tanh\left(\mathbf{q}^\top \mathbf{W}_q\right)$ and $\tilde{\mathbf{v}} = \tanh\left(\mathbf{v}^\top \mathbf{W}_v\right)$, $\mathcal{T}_c$ is the fusion weight tensor, and $\mathbf{W}_o$ is the output processing weight matrix. Moreover, MUTAN [39] adopts a low rank for the fusion weight tensor $\mathcal{T}_c$, as follows:

$$\mathcal{T}_c[:,:,k] = \sum_{r=1}^{R} \mathbf{m}_r^k \circ \mathbf{n}_r^{k\top}, \tag{25}$$

where $\mathbf{m}_r^k$ and $\mathbf{n}_r^{k\top}$ are weight vectors and R is the number of ranks. In this way, MUTAN can represent comprehensive bilinear interactions while maintaining a reasonable model size by factorizing the interaction tensors into interpretable elements.

Furthermore, compact trilinear interaction (CTI) [40] was proposed, which uses an attention-like structure. Instead of presenting the given data as a single vector, this method represents every modality as a matrix $\mathbf{A} \in \mathbb{R}^{n_1 \times d_a}$, where d_a corresponds to the feature dimension and n_1 denotes the number of states. The CTI simultaneously learns high-level trilinear joint representations in VQA tasks and overcomes both the computational complexity and memory issues in trilinear interaction learning [40].

3.3 Multi-way Data Compression

TNNs present a powerful framework for addressing the unique challenges in multi-dimensional data compression. Unlike traditional compression methods that treat data as vectors or matrices, or conventional tensor methods that rely on fixed decomposition structures, TNNs leverage learnable neural architectures to adaptively preserve and exploit the natural multi-dimensional relationships in the data, leading to more efficient and accurate representations with theoretical guarantees.

The BNTD [41] first introduces TNNs and advances multi-way data compression through a principled probabilistic framework,

effectively modelling complex entity-relation interactions and incorporating prior information via neural tensor architectures. The FLEST leverages tensor factorization and embedding matrix decomposition as a data compression mechanism to enable efficient federated knowledge graph completion while preserving privacy in distributed settings. The TTHRESH method [46] leverages HOSVD decomposition combined with bit-plane, run-length and arithmetic coding to efficiently compress high-dimensional gridded data for visualization, achieving smooth quality degradation and enabling low-cost compressed-domain manipulations while providing competitive compression ratios at low-to-medium bit rates. Fan *et al.* [47] proposed a multi-mode deep matrix and tensor factorization approach (M2DMTF) that employs TKD with factor matrices generated using multilayer perceptrons, effectively handling complex tensor data with missing values and noise. Lee and Shin [48] developed a robust factorization method specifically designed for real-world tensor streams containing patterns, missing values, and outliers. Lamba *et al.* [49] introduced a method for incorporating side information into tensor factorization, improving the quality of compression and representation learning.

NeuKron [44] extends tensor neural networks by introducing auto-regressive neural networks to generalize Kronecker products, enabling constant-size lossy compression of sparse reorderable matrices and tensors. TensorCodec [42], [43] extends tensor neural networks for efficient data compression by introducing neural tensor-train decomposition, tensor folding, and mode-index reordering techniques, enabling accurate compression without strong data assumptions. Light-IT and Light-IT++ [42] extend tensor neural networks for efficient data compression by introducing vocabulary-based compression and core tensor operations, enabling compact and accurate representation of irregular tensors. The TT-PC method [45] introduces a novel TNN for efficient point cloud representation and fast approximate nearest-neighbour search, demonstrating the superior performance of TNNs in both anomaly detection and vector retrieval tasks through its probabilistic compression approach and inherent hierarchical structure.

3.4 Multi-task Data Training

For multitask learning applications, WISDOM [62] pioneered an incremental learning algorithm that performs supervised tensor decomposition on spatio-temporal data encoded as third-order tensors, simultaneously training spatial and temporal prediction models from extracted latent factors while incorporating domain knowledge, demonstrating superior performance over baseline algorithms in global-scale climate data prediction across multiple locations. Yang *et al.* [51] then proposed the Tensor Train multitask (TTMT) and Tucker multitask (TMT) models using TT and Tucker formats, respectively, to alleviate the negative transfer problem in a hard sharing architecture and reduce the parameter volume in a soft structure. The M2TD method [57] stitches patterns from partitioned parameter subspaces of large simulation ensembles to efficiently discover underlying dynamics and interrelationships while maximizing accuracy under limited simulation budgets. Zhang *et al.* [54] proposed a tensor network-based multi-task model that decomposes person Re-ID into camera-specific classification tasks and leverages low-rank tensor decomposition to capture cross-camera correlations while aligning feature distributions across different views. The SMART method [227] decomposes spatio-temporal data into interpretable latent factors and trains an ensemble of spatial-temporal predictors while incorporating domain constraints to handle large-scale spatio-temporal prediction tasks efficiently. A PEPS-like concatenated TN layer [52] for multitask missions was also proposed, which, unlike the TTMT and TMT models that suffer from the negative transfer problem due to their hard sharing architectures, only contains a soft sharing layer, thereby achieving better performance. The MTCN method [53] achieves superior face multi-attribute prediction by sharing all features in lower layers while differentiating attribute features in higher layers, by incorporating tensor canonical correlation analysis to exploit inter-attribute relationships. The CTNN method [56] combines depthwise separable CNN and low-rank tensor networks to efficiently extract both local and global features from multi-task brainprint data, achieving high recognition accuracy with limited training samples while providing interpretable channel-specific biomarkers. The GTTN method [55] combines matrix trace norms from all possible tensor flattenings to automatically discover comprehensive low-rank structures in deep multi-task learning models, eliminating the need for manual specification of component importance. Zhang *et al.* [58] propose a tensor-based multi-task learning framework that leverages spatio-temporal similarities between brain biomarkers to predict Alzheimer's disease progression by encoding MRI morphological changes into a third-order tensor and extracting shared latent factors through tensor decomposition.

More recently, FTN [59] efficiently adapts a frozen backbone network to multiple tasks/domains by adding task-specific low-rank tensor factors, achieving comparable accuracy to independent single-task networks while requiring significantly fewer additional parameters and preventing catastrophic forgetting. The MULTIPAR method [60] extends PARAFAC2 with multi-task learning capabilities for EHR mining, yielding improved phenotype extraction and prediction performance through joint supervision of static and dynamic tasks. The MMER-TD method [63] combines tensor decomposition fusion and self-supervised multi-task learning, employing Tucker decomposition to reduce parameters and prevent overfitting, while building a dual learning mechanism for multimodal and unimodal tasks with label generation to capture inter-modal emotional variations. Liu *et al.* [61] map speech quality features into higher-dimensional space through tensor network, enabling improved feature correlation analysis and mean opinion score prediction, while a novel loss function simultaneously optimizes regression, classification, and correlation metrics.

3.5 Quantum (State-based) Data Representation

To process machine learning tasks in a quantum system, the input data should be converted into a linear combination of some quantum states as an orthogonal basis; in the form

$$
\begin{aligned}
&|\psi\rangle = \sum_{d_1 \dots d_N = 1}^{M} \mathcal{A}_{d_1 \dots d_N} \, |\psi_{d_1}\rangle \circ \cdots \circ |\psi_{d_N}\rangle \,, \\
&s.t \quad \sum_{d_1 \dots d_N = 1}^{M} \mathcal{A}^2_{d_1 \dots d_N} = 1, \quad \mathcal{A}_{d_1 \dots d_N} \geq 0,
\end{aligned} \tag{26}
$$

where $|\cdot\rangle$ is the Dirac notation of a vector with complex values [228], and $\circ$ denotes the outer product operation. The tensor $\mathcal{A}$ is the combination coefficient tensor and is always represented and analyzed via a low-rank TN [10]. To embed classic data into a quantum state for adapting quantum systems, Stoudenmire and

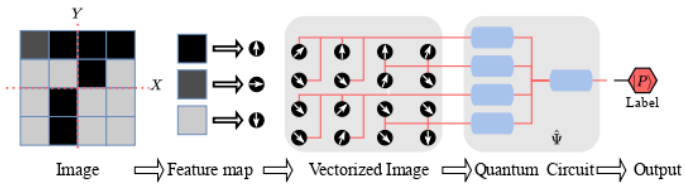


Fig. 7: The processing procedure employed for quantum embedded data [229]. Quantum circuits can be simulated via TNNs on classic electronic computers, and some special TNNs (such as ConvAC) can also be theoretically implemented on a realistic quantum circuit.

Schwab [64] proposed a quantum state mapping function $\phi^i(x_i)$ for the i-th pixel x_i in a grayscale image as

$$\phi^i(x_i) = [\cos(\frac{\pi}{2}x_i), \sin(\frac{\pi}{2}x_i)]. \tag{27}$$

The values of pixels are transformed into the range from 0.0 to 1.0 via the mapping function. Furthermore, a full grayscale image $\mathbf{x}$ can be represented as outer products of the mapped quantum states of each pixel as

$$\Phi^{1,2,\dots N}(\mathbf{x}) = \phi^1(x_1) \circ \phi^2(x_2) \circ \cdots \phi^N(x_N), \tag{28}$$

where $\Phi^{1,2,\dots,N}(\mathbf{x}) \in \mathbb{R}^{\overbrace{2 \times 2 \cdots \times 2}^{N}}$. Through Eq. (28), it is feasible to associate realistic images with real quantum systems.

For a natural language document, the i-th word $|x_i\rangle$ can also be represented as the sum of orthogonal quantum state bases $|\phi_{h_i}\rangle$ $(h_i = 1, \dots, M)$ [65], [66], [67], [68] corresponding to a specific semantic meaning M as

$$\begin{aligned} |x_i\rangle &= \sum_{h_i=1}^{M} \alpha_{i,h_i} |\phi_{h_i}\rangle, \\ s.t \quad &\sum_{h_i=1}^{M} \alpha_{i,h_i}^2 = 1, \quad \alpha_{i,h_i} \geq 0, \end{aligned} \tag{29}$$

where α_{i,h_i} is the associated combination coefficient for each semantic meaning. The constraint of α_i ensures the quantum state normalization and non-negativity of the coefficients, which follows the rules of quantum mechanics. After completing data mapping, the embedded quantum data can be processed by TNNs on a realistic quantum circuit, as shown in Fig. 7. The loss functions of TNNs can also be defined through the properties of quantum circuits. Such a procedure can be simulated on classic electronic computers via TNs and can be theoretically efficiently implemented on realistic quantum systems.

4 Sustainable AI through TNNs: Compact Model Structures

DNNs have extraordinarily high spatial and temporal complexity levels, as deeply stacked layers contain large-scale matrix multiplications. As a result, DNNs usually require several days for training while occupying a large amount of memory for inference purposes. In addition, large weight redundancy has been proven to exist in DNNs [230], indicating the possibility of compressing DNNs while maintaining performance. Motivated by this, a wide range of compression techniques have been developed, including pruning [231], quantization [232], distillation [233] and low-rank decomposition [85]. Among them, applying TNs to DNNs to construct TNNs can be a good choice since TNNs have excellent abilities to approximate the original weights with much fewer parameters [131]. In this direction, researchers have completed many studies, especially concerning the reconstruction of convolutional and fully connected layers through a variety of TD formats [71], [76], [85], [234]. With compact architectures, these TNNs can achieve improved performance with less redundancy. In this section, we examine how TNNs enable more sustainable AI through compact model structures. Modern DNNs, while powerful, often require extensive computational resources and memory due to their deep architectures and large-scale matrix operations. Additionally, studies have shown significant weight redundancy in DNNs, suggesting opportunities for compression without sacrificing performance. Here, we explore five key TNN architectures that address these challenges: TCNNs (Section 4.1), TRNNs (Section 4.2), tensorial Transformers (Section 4.3), TGNNs (Section 4.4), and tensorial quantum neural networks (Section 4.5). By leveraging tensor decomposition techniques and efficient parameter sharing, these approaches achieve significant model compression while maintaining or even enhancing performance compared to their conventional counterparts. We also examine the emerging applications of tensor networks in large language models (Section 4.6), where they enable efficient compression and parameter-efficient fine-tuning.

4.1 TCNNs

CNNs have recently achieved much success. However, the enormous sizes of CNNs cause weight redundancy and superfluous computations, affecting both their performance and efficiency. Indeed, TD methods can be effective solutions to this problem, and CNNs represented with tensor formats are called TCNNs. Prior to introducing TCNNs, we formulate a vanilla CNN, shown in Fig. 3 (a), as

$$\mathcal{Y} = \mathcal{X} \circledast \mathcal{C} + \mathbf{b}, \tag{30}$$

where $\mathcal{C} \in \mathbb{R}^{K \times K \times I \times O}$ denotes a convolutional weight, $\mathcal{X} \in \mathbb{R}^{I \times H \times W}$ denotes an input, $\mathcal{Y} \in \mathbb{R}^{O \times H' \times W'}$ denotes an output, $\mathbf{b} \in \mathbb{R}^{O}$ represents a bias, and $\circledast$ denotes a convolutional operator. K represents the kernel window size, I is an input channel, H and W denote the height and width of $\mathcal{X}$, O is an output channel, and H' and W' denote the height and width of $\mathcal{Y}$, respectively. TCNNs mainly focus on decomposing the channels I and O. In detail, the weight $\mathcal{C}$ is first reshaped to $\tilde{\mathcal{C}} \in \mathbb{R}^{K \times K \times I_1 \times I_2 \times \dots I_M \times O_1 \times O_2 \times \dots O_N}$, where $\prod_{k=1}^{M} I_k = I$ and $\prod_{k=1}^{N} J_k = J$. Then, TCNNs can be derived by tensorizing the reshaped convolutional kernel $\tilde{\mathcal{C}}$.

To accelerate the CNN training and inference process, CP-CNN [69], [70], [71], [72], [73] is constructed by decomposing the convolutional weight into the CP format, as shown in Fig. 3 (d). CP-CNN only contains vectors as subcomponents, leading to an extremely compact structure and the highest compression ratio. As with CP-CNN, it is possible to implement additional TCNNs by applying tensor formats (as seen in the examples in Fig. 2) to the convolutional weight. Tucker decomposition, a widely used tensor format, is often applied to CNNs to form Tucker-CNNs [74], [75]. Different from simple Tucker formats, a BTT-CNN has a hyperedge R_c, which can denote the summation of Tucker decompositions. Other BTT-CNNs [76] have also been proposed. Compared to Tucker CNNs, BTT-CNNs are much more powerful and usually derive better results [76]. Highly compact TT formats have also

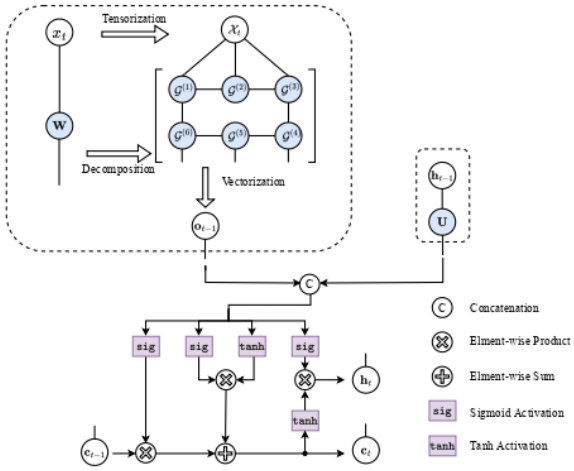


Fig. 8: The tensor ring LSTM. It is effective at reducing the parameters of an LSTM model by replacing the input-to-hidden transformation weights with TR decomposition.

been introduced to CNNs to implement TT-CNNs [77], [78], [79]. Compared to TTs, TR formats are usually much more compact [80], and TR-CNNs [80] are much more powerful than TT-CNNs. To address the degenerate problem in tensorial layers, a stable decomposition method CPD-EPC [72] is proposed with a minimal sensitivity design for both CP convolutional layers and hybrid Tucker2-CP convolutional layers. The TR-Compress method [82] extends tensor networks through tensor ring decomposition to optimize neural network compression, enabling efficient parameter reduction while preserving model accuracy through optimized factorization and execution scheduling.

There are also some tensorial convolutional neural networks that decompose more than just the convolution cores. The tensorized network (T-Net) [81] treats the whole network as a one-layer architecture and then decomposes it. As a result, the T-Net achieves better results with a lighter structure. The CP-higher-order convolution (CP-HOConv) [83] utilizes the CP format to handle tasks with higher-order data, e.g., spatiotemporal emotion estimation.

4.2 Tensor Recurrent Neural Networks

RNNs, such as the vanilla RNN and LSTM, have achieved promising performance on sequential data. However, when dealing with high-dimensional input data (e.g., video and text data), the input-to-hidden and hidden-to-hidden transformations in RNNs will result in high memory usage rates and computational costs. To solve this problem, low-rank TD is efficient for compressing the transformation process in practice. First, we formulate an RNN as

$$\mathbf{h}^{(t+1)} = \phi(\mathbf{W}\mathbf{x}^{(t)} + \mathbf{U}\mathbf{h}^{(t)} + \mathbf{b}), \tag{31}$$

where $\mathbf{h}^{(t)} \in \mathbb{R}^{O}$ and $\mathbf{x}^{(t)} \in \mathbb{R}^{I}$ denote the hidden state and input feature at time t, respectively, $\mathbf{W} \in \mathbb{R}^{O\times I}$ is the input-to-hidden matrix, $\mathbf{U} \in \mathbb{R}^{O\times O}$ represents the hidden-to-hidden matrix, $\mathbf{b} \in \mathbb{R}^{O}$ is a bias, while $\phi(\cdot)$ indicates a series of operations that form RNN variants, including the vanilla RNN and LSTM [235]. Eq. (31) can also be reformulated in a concatenated form that is widely used in TD, given by

$$\mathbf{h}^{(t+1)} = \phi([\mathbf{W}, \mathbf{U}][\mathbf{x}^{(t)}, \mathbf{h}^{(t)}] + \mathbf{b}), \tag{32}$$

where $[\mathbf{W}, \mathbf{U}] \in \mathbb{R}^{O\times(I+O)}$ and $[\mathbf{x}^{(t)}, \mathbf{h}^{(t)}] \in \mathbb{R}^{(I+O)}$ denote the concatenation of $\mathbf{W}, \mathbf{U}$ and $\mathbf{x}^{(t)}, \mathbf{h}^{(t)}$, respectively. As shown in Fig. 8, there are usually two ways to decompose RNNs: (a) only tensorizing $\mathbf{W}$, which is often the largest component in an RNN, and (b) tensorizing $[\mathbf{W}, \mathbf{U}]$ for extreme compression. Note that since $\mathbf{U}$ is usually smaller than $\mathbf{W}$, no works decompose $\mathbf{U}$ only. The process of implementing a TRNN is the same as that used to implement a TCNN, namely, by reshaping the weights into higher-order formulations and replacing them with tensor formats.

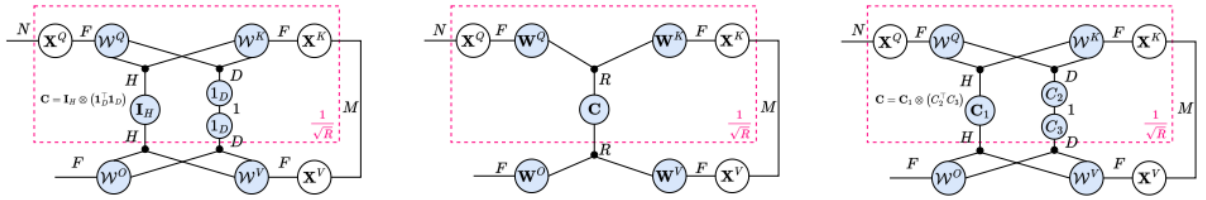
The most direct and simple compression method is to solely decompose the enormous input-to-hidden matrix $\mathbf{W}$. The CP-RNN and Tucker-RNN [74] can be directly constructed with the CP and Tucker formats, respectively. With an extremely compact low-rank structure, the CP-RNN can always achieve the smallest size in comparison with other tensor formats. The TT-RNN [84] implements the TT format on an RNN to obtain a high parameter compression ratio. However, the TT-RNN suffers from a linear structure with two smaller endpoints, which hinders the representation ability and flexibility of TT-based models. To release the power of a linear architecture, TRs were proposed to link the endpoints to create a ring format [25]. The TR-An RNN [85] with a TR was formed to achieve a much more compact network. The BTT-RNN [76], [86] was constructed on the generalized TD approach, the BTT decomposition [236]. The BTT-RNN can automatically learn interparameter correlations to implicitly prune redundant dense connections and simultaneously achieve better performance.

Moreover, studies are utilizing TD to compress an RNN's two transformation layers, and some have even developed decomposition methods that are suitable for both RNNs and CNNs. The TT-GRU [87] and the HT-RNN [88] methods decompose $[\mathbf{W}, \mathbf{U}]$ to attain a higher compression ratio. Specifically, TT-GRU [87] applies a TT for decomposition, and the HT-RNN [88] adopts HT decomposition. Unlike prior works that decompose hidden matrices, Conv-TT-LSTM [90] utilizes the idea of a TT to represent convolutional operations. As shown in Fig. 8, through a TT-like convolution, Conv-TT-LSTM can replace convolutional LSTM with fewer parameters while achieving good results on action benchmarks. For the adaptation of both CNNs and RNNs, a hybrid TD (termed HT-TT) method that combines HT and TT decomposition [89] was adopted to compress both the CNN and RNN $[\mathbf{W}, \mathbf{U}]$ matrices. The MPS-NLP [92] proposes tensor recurrent neural networks (TRNNs) through matrix product states and entanglement entropy, enabling explainable natural language processing while maintaining model performance. In addition, the tensor contraction layer (TC-Layer) [91] was designed to replace the fully connected layer and therefore can be utilized as the last layer of a CNN and the hidden layers in RNNs. Interestingly, TC-Layer is a special case of a TT-based layer obtained by setting the ranks to 1.

4.3 Tensorial Transformers

Transformers [179], [237] are well known for processing sequence data. Compared with CNNs and RNNs, Transformers can be stacked into large-scale sizes to achieve significant performance gains [180]. However, Transformers are still redundant, similar to classic DNNs, and can be made smaller and more efficient [95]. Therefore, TD, as a flexible compression tool, can be explored to reduce the numbers of parameters in Transformers [93], [99], [238].

Classic Transformers mainly consist of the self-attention (SA) mechanism and feedforward Networks (FFNs). The SA processes



(a) Multi-Head Self-Attention (MHSA). (b) Tunable-Head Self-Attention (THSA). (c) The design space of $\mathbf{C}$.

Fig. 9: Tensor diagrams for SA modules [93]. (a) It is feasible to represent a classic multihead SA (MHSA) mechanism as a tensor diagram. MHSA can be treated as a special case of tunable-head self-attention (THSA) by setting $\mathbf{C} = \mathbf{I}_H \otimes (\mathbf{1}_D^\top \mathbf{1}_D)$. (b) The THSA of the Tuformer can be a more generalized version of SA through a trainable matrix $\mathbf{C}$. (c) THSA has a design space formulated as $\mathbf{C} = \mathbf{C}_1 \otimes (C_2^\top C_3)$, which is the direct generalized form of MHSA.

the given query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$ and value matrix $\mathbf{V}$ with parameters $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O$. More generally, SA is separated into n heads: $\{\mathbf{W}_i^Q\}^n, \{\mathbf{W}_i^K\}^n, \{\mathbf{W}_i^V\}^n, \{\mathbf{W}_i^O\}^n$. Each head can be calculated as

$$\mathrm{Att}_i(\mathbf{Q},\mathbf{K},\mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{W}_i^Q \mathbf{W}_i^{K^T} \mathbf{K}^T}{\sqrt{d}}\right) \mathbf{V}\mathbf{W}_i^V \mathbf{W}_i^{O^T}. \tag{33}$$

Then, $\mathrm{SA}((\mathbf{Q},\mathbf{K},\mathbf{V})) = \sum_{i=1}^n \mathrm{Att}_i(\mathbf{Q},\mathbf{K},\mathbf{V})$. Another important component, the FFN, is formulated as

$$\mathrm{FFN}(\mathbf{X}) = \mathrm{ReLU}(\mathbf{X}\mathbf{W}^{in} + \mathbf{b}^{in})\mathbf{W}^{out} + \mathbf{b}^{out}, \tag{34}$$

where $\mathbf{X}$ is the input, $\mathbf{b}^{in}$ and $\mathbf{b}^{out}$ are biases, and $\mathbf{W}^{in}$ and $\mathbf{W}^{out}$ are weights. The number of parameters in a Transformer is mainly based on its linear transformation matrices, i.e., $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O, \mathbf{W}^{in}$ and $\mathbf{W}^{out}$.

Therefore, most compression studies focus on eliminating the parameters of these matrices. For instance, the MPO structure was proposed to decompose each matrix in a Transformer [93], generating central tensors (containing the core information) and small auxiliary tensors. A tuning strategy was further adopted to continue training the auxiliary tensors to achieve a performance improvement while freezing the weight of the central tensor to retain the main information of the original matrix. Moreover, observing that a low-rank MPO structure can cause a severe performance drop, Hypoformer [94] was proposed based on hybrid TT decomposition; this approach concatenates a dense matrix part with a low-rank MPO part. Hypoformer retains the full-rank property while reducing the required numbers of operations and parameters to compress and accelerate the base Transformer. In addition, by concatenating all matrices into one larger tensor, Tucker-Bert [95] decomposes the concatenated tensor with Tucker decomposition to greatly reduce the number of parameters, leading to extreme compression and maintaining comparably good results. Compared to compressing the original attention operation, multiway multimodal transformer (MMT) [96] explores a novel generalized tensorial attention operation to model modality-aware multiway correlations for multimodal datasets. The TCTN (Tensor Compressed Transformer Network) [97] extends tensor networks through tensor train decomposition to compress traffic forecasting transformers, achieving efficient parameter reduction while maintaining prediction accuracy through optimized spatial-temporal modeling. Interestingly, Tuformer [99] generalizes MHSA into the Tucker form, thus containing more expressive power and achieving better results, as shown in Fig. 9. Advances in tensorial causal learning have emerged through the implementation of causal capsules and tucker-format tensor transformers for latent variable interaction control. Recently, T6 [98], a novel Transformer architecture leveraging Tensor Product Attention (TPA), compresses KV cache through tensor decomposition to handle longer sequences and outperforms existing attention mechanisms like MHA, MQA, GQA, and MLA on language modeling tasks.

4.4 TGNNs

Graph Neural Networks (GNNs) have achieved groundbreaking performances across a range of applications and domains [239]. One classic GNN layer consists of an aggregation function for aggregating the neighbor node information and an update function for updating the current node information. For example, the processing step for node v in the k-th layer of a GNN can be formulated as

$$\begin{aligned} \mathbf{a}_v^{(k)} &\leftarrow \mathrm{Aggregate}_{(k)}\left(\left\{\mathbf{h}_u^{(k-1)}, \forall u \in \mathcal{N}(v)\right\}\right), \\ \mathbf{h}_v^{(k)} &\leftarrow \mathrm{Update}_{(k)}\left(\mathbf{h}_v^{(k-1)}, \mathbf{a}_v^{(k)}\right), \end{aligned} \tag{35}$$

where $\mathbf{a}_v^{(k)}$ is an aggregated embedding vector, $\mathbf{h}_v^{(k-1)}$ is a node embedding vector, and $\mathcal{N}(v)$ is a neighbor node set. A typical choice for the update function is a simple one-layer perceptron, and simple summation/maximization is always chosen as the aggregation function. Classic GNNs suffer from low model expressivity since high-order nonlinear information among nodes is missed [101]. Because of the merits of the tradeoff between expressivity and computing efficiency, the usage of TGNNs for graph data processing is quite beneficial.

To efficiently parameterize permutation-invariant multilinear maps for modeling the interactions among neighbors in an undirected graph structure, a TGNN [101] makes use of a symmetric CP layer as its node aggregation function. It has been demonstrated that a TGNN has a strong capacity to represent any multilinear polynomial that is permutation-invariant, including the sum and mean pooling functions. Nimble GNN [103] innovatively applies tensor-train decomposition to GNN embeddings with graph-aware tensor operations, achieving up to 81,362× compression while maintaining accuracy. Compared to undirected graph processing, TGNNs are more naturally suited for high-order graph structures, such as knowledge graphs or multi-view graph. Traditional relational graph convolutional networks neglect the trilinear interaction relations in knowledge graphs and additively combine the information possessed by entities. The TGCN [102] was proposed by using a low-rank Tucker layer as the aggregation function to improve the efficiency and computational space requirement of multilinear modeling. The RTGNN [104], which applies a Tucker format

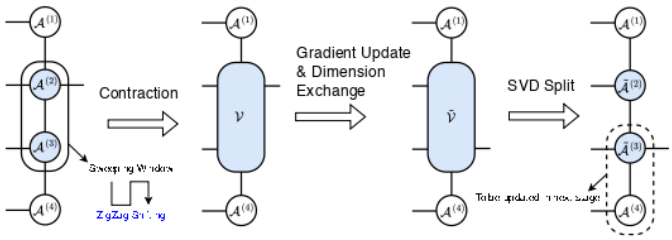


Fig. 10: A single stage of the Sweeping method [64]. In each stage, Sweeping only updates the nodes in the sweeping window, which shifts along a zigzag trajectory.

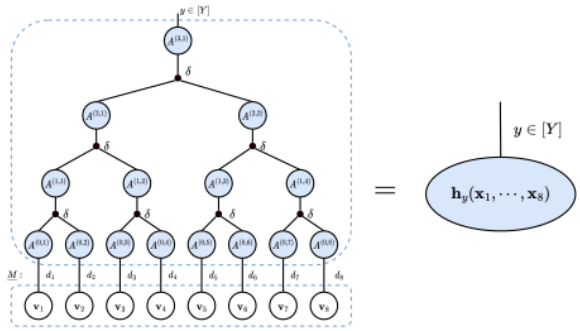


Fig. 11: ConvAC is equivalent to an HT-like TN [241]. $\mathbf{x}_1, \ldots, \mathbf{x}_N \in \mathbb{R}^s$, where $\mathbf{x}_1$ corresponds to a local patch from the input image or the feature map, and $v^{(0,j)}$ is the linear transformation of $\mathbf{x}_j$. The width is 2 for a single block. Notably, a single layer is equivalent to a CP format.

structure to extract the graph structure features in the common feature space, was introduced to capture the potential high order correlation information in multi-view graph learning tasks. TGNNs are also appropriate for high-order correlation modeling in dynamic spatial-temporal graph processing situations. For example, the DSTGNN [106] applies learnable TTG and STG modules to find dynamic time relations and spatial relations, respectively. Then, the DSTGNN explores the dynamic entangled correlations between the STG and TTG modules via a PEPS layer, which reduces the number of DSTGNN parameters.

4.5 Tensorial Quantum Neural Networks

Quantum neural networks aim to process quantum data directly in quantum systems. One representative work bridging TNNs with quantum data processing is the MPS-based architecture proposed by Stoudenmire and Schwab [64], which formulates the classification task of quantum mapped image data (as introduced in Sec 3.5) as optimizing functions indexed by labels, given by

$$c = \frac{1}{2} \sum_{n=1}^{N_T} \sum_{\ell} \left(f^{\ell}(\mathbf{x}_n) - \mathbf{y}_{n\ell} \right)^2, \tag{36}$$

where N_T denotes the number of training samples, and $\mathbf{y}_n$ denotes the true one-hot label vector of $\mathbf{x}_n$. The optimization process is carried out to minimize this cost function in stages with stochastic gradient descent. A single stage is shown in Fig. 10. In each stage, two MPS tensors $\mathcal{A}^{(2)}$ and $\mathcal{A}^{(3)}$ are combined into a single bond tensor $\mathcal{V}$ via tensor contraction. Then, the tensor $\mathcal{V}$ is updated with gradients. Finally, $\tilde{\mathcal{V}}$ is decomposed back into separate tensors with the SVD algorithm. This work establishes a crucial connection between quantum physics and machine learning: the MPS structure, originally developed for quantum many-body systems, naturally bridges quantum-inspired tensor methods with neural architectures, where the bond dimensions serve as model complexity controls. The Sweeping method demonstrates how quantum-inspired optimization techniques can be effectively adapted for machine learning tasks. Furthermore, this framework's extensibility to other tensor network structures like PEPS [240] suggests its potential for advancing both quantum and classical architectures while maintaining computational tractability.

The expressive power of previously developed quantum data processing models, e.g., the MPS models [122] and the Born machine [123], suffers from a lack of nonlinearity. Classic nonlinear operators, e.g., activation functions (such as the rectified linear unit (ReLU) function) and average/max pooling, can significantly benefit model performance. However, classic nonlinearity cannot be directly implemented in a quantum circuit. To solve this problem, the ConvAC network [124], [125] was proposed to adopt quantum deployable product pooling as a nonlinear operator, proving that ConvAC can be transformed into ConvNets with ReLU activations and average/max pooling. The whole structure of ConvAC can be represented by an HT format and has been proven to be theoretically deployable in realistic quantum systems.

A tensor diagram example of ConvAC is shown in Fig. 11, where one hidden layer of ConvAC is in a CP format. ConvAC can also handle language data [67] by mapping natural language sentences into quantum states via Eq. (29). ConvAC is a milestone in that deep convolutional networks, along with nonlinear modules, are implemented on quantum circuits. It serves as an inspiration for the integration of more NNs into quantum systems. This has led to several important developments. First, Zhang *et al.* [126] introduced the tensor space language model (TSLM), which generalizes the n-gram language model. Building on this, ANTN (Autoregressive Neural TensorNet) [127] bridges tensor networks and autoregressive neural networks through matrix product states, enabling efficient quantum many-body simulation while preserving both physical prior and model expressivity.

More recently, ADTN [128] extends quantum tensor networks through deep tensor decomposition to compress neural networks, achieving quantum-inspired exponential parameter reduction while improving model accuracy. Further advancing this direction, TTLM [129] extends tensor networks through tensor train decomposition to enable language modeling, achieving efficient sequence modeling through recurrent parameter sharing while preserving model expressivity. Tensor Network Functions (TNFs) [130] offers a novel perspective on tensor networks by enabling efficient computation of strict variational energies, representation of volume law behavior, and mapping of neural networks and quantum states while removing traditional computational restrictions on tensor network contractions.

4.6 Tensor Networks in Large Language Models

With the recent surge of large language models (LLMs), tensor networks have emerged as a powerful framework for compressing and accelerating these massive models through various decomposition techniques and parameter-efficient fine-tuning approaches. TensorGPT [107] extends tensor neural networks to efficiently compress large language models through tensor-train decomposition, enabling training-free compression of token embeddings into lower-

dimensional matrix product states. CompactifAI [108] leverages quantum-inspired tensor networks to achieve extreme compression of large language models through efficient correlation truncation in the model's tensor space and controllable tensor network decomposition. FASTER-LMs [109] extends tensor networks through canonical tensor decomposition to accelerate language model inference, enabling efficient multi-token prediction while preserving dependencies between predicted tokens. The TQCompressor [111] enhances tensor networks through permutation-based Kronecker decomposition for neural network compression, achieving improved model expressivity while reducing parameter count. The TTM [110] harnesses tensor networks through tensor train matrix decomposition to enable efficient pre-training of GPT models, achieving 40% parameter reduction.

Additionally, tensor networks have also demonstrated significant success in parameter-efficient fine-tuning approaches like LoRA, leading to various innovative adaptations. The TT-LoRA [112] extends tensor networks for parameter-efficient fine-tuning of large language models by leveraging tensor train decomposition, enabling extreme model compression while maintaining model accuracy. The SuperLoRA [113] extends tensor networks through tensor decomposition and Kronecker products to unify and enhance low-rank adaptation methods, enabling highly parameter-efficient fine-tuning of large vision models. The Quantum-PEFT [114] adapts quantum tensor networks for parameter-efficient fine-tuning by leveraging quantum unitary parameterization and Pauli rotation, enabling logarithmic parameter scaling while maintaining model performance. The QuanTA [119] utilizes tensor networks through quantum-inspired circuit structures to enable efficient high-rank fine-tuning, providing a theoretically-grounded alternative to traditional low-rank adaptation methods while maintaining parameter efficiency and model performance. The LoRA-PT [115] extends tensor networks through tensor singular value decomposition to enable parameter-efficient fine-tuning, leveraging principal tensor components for efficient neural network adaptation. The FLoRA [116] employs tensor networks through Tucker decomposition to enable parameter-efficient fine-tuning for N-dimensional parameter spaces, maintaining structural integrity while achieving low-rank adaptations. The LoTR [117] leverages tensor networks through Tucker decomposition to enable weight adaptation of neural networks, achieving parameter-efficient fine-tuning while preserving tensor structure. The Quantum-inspired-PEFT [118] extends tensor networks through subspace-based geometric transformations to achieve parameter-efficient model adaptation, enabling unified interpretation of matrix and tensor factorizations. The DoTA [121] utilizes MPO of pre-trained weights for tensor networks based on fine-tuning, improving upon random initialisation methods by better capturing high-dimensional structures while achieving comparable performance with fewer parameters. The FacT [120] leverages tensor networks through tensorization-decomposition to enable efficient fine-tuning of vision transformers, performing tensor low-rank adaptation while maintaining cross-layer structural information.

Remark. Compact TNNs have demonstrated the potential to achieve extremely high compression ratios while preserving their model performance. However, their computational acceleration rates are not very significant compared with their compression ratios, which is mainly due to the contraction operations. This therefore calls for further research to improve the employed contraction strategies, since unoptimized contraction strategies can result in unsatisfactory running memory consumption.

5 Training Strategies for TNNs

While the aforementioned TNNs can perform well on various tasks and machines, it is also worth exploring training strategies with more stability, better performance and higher efficiency. In this section, we introduce such strategies in three groups: (1) strategies for stabilizing the training processes of TNNs are presented in Section 5.1, (2) strategies for selecting and searching the ranks of TNNs are provided in Section 5.2, and (3) strategies for applying hardware speedup are shown in Section 5.3.

5.1 Stable Training Approaches

Despite their success, TNNs face significant training challenges stemming from their inherent multilinear characteristics. While traditional neural networks primarily rely on simple linear operations like matrix multiplication, TNNs involve tensor contractions that result in exponentially scaling data flows as the number of modes increases linearly [132]. This exponential scaling affects both the forward propagation of features and the backward propagation of gradients, creating substantial computational and numerical stability challenges. Several approaches have been proposed to address these issues. One straightforward solution involves using full-precision float64 format to represent large weights, which helps mitigate numerical instability problems. However, this approach comes with significant drawbacks - the higher precision format requires more computational resources and increases processing time compared to lower-precision alternatives like float16. Conversely, while lower precision formats offer computational efficiency, they can introduce numerical stability issues that compromise training effectiveness. To balance these competing concerns, Panagakis *et al.* [131] introduced an innovative mixed-precision strategy. This dynamic precision approach adaptively adjusts numerical precision during different phases of computation, effectively creating a trade-off between computational efficiency and numerical stability. By selectively applying higher precision only where necessary, this strategy successfully reduces memory requirements while maintaining training stability. This approach has proven particularly effective in handling the complex tensor operations characteristic of TNNs, enabling more efficient and reliable training processes. MANGO [133] accelerates large model training by establishing comprehensive linear correlations between all weights of pretrained and target models, rather than using partial weight mapping as in previous approaches like bert2BERT and LiGO. As shown in Figure 12, MANGO operates on the entire Transformer structure, including Multi-Head Self-Attention blocks, Feed-Forward Networks, and normalization layers, applying its full mapping operator to correlate parameters between small and large models through TR-MPO.

Another feasible way to solve the training problem lies in developing a suitable initialization method for tensor neural networks (TNNs). Currently, two widely adopted adaptive initialization methods in deep learning are Xavier [244] initialization and Kaiming [245] initialization. Xavier initialization, proposed by Glorot and Bengio in 2010, regulates the variances of data flows between layers to prevent the vanishing gradient problem in deep networks. Similarly, Kaiming initialization, introduced by He *et al.* in 2015, was specifically designed for networks using ReLU activation functions. However, these conventional initialization methods face two major challenges when applied to TNNs. First, they cannot accurately calculate the appropriate scales for TNNs due to their inability to account for the complex interactions

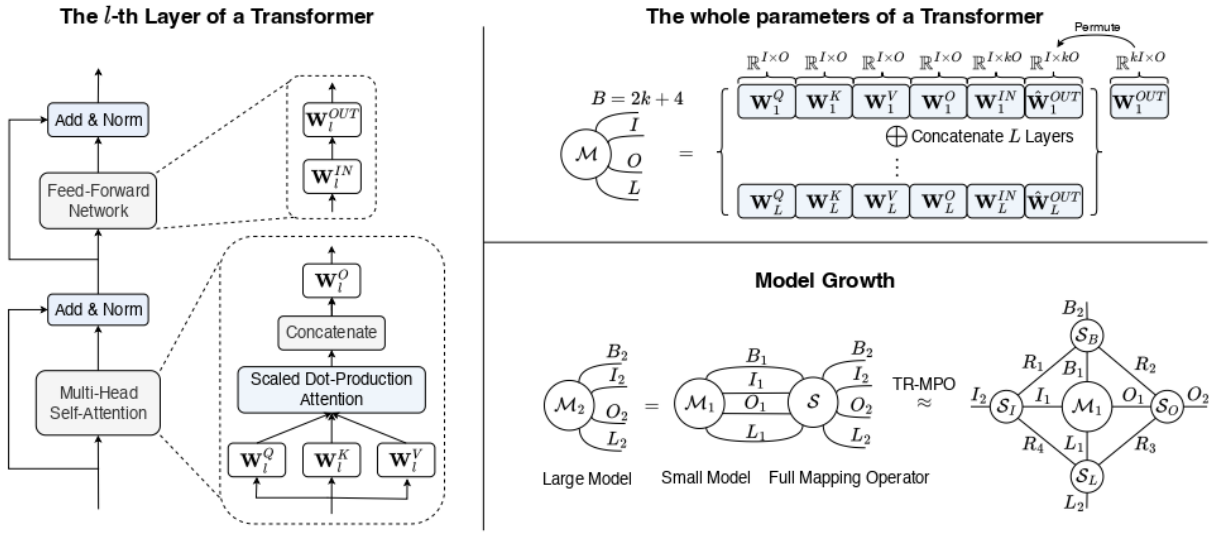


Fig. 12: The architecture of MANGO, showing how it establishes comprehensive linear correlations across the entire Transformer structure, including Multi-Head Self-Attention (MHSA), Feed-Forward Networks (FFN), and normalization layers. The full mapping operator leverages TR-MPO to correlate parameters between small and large models.

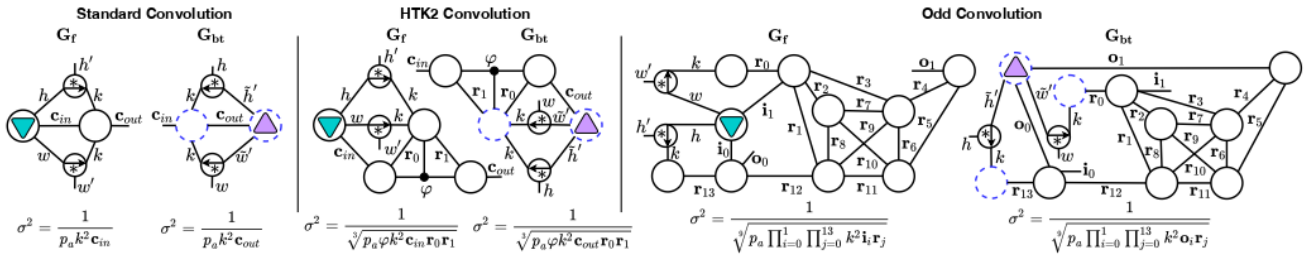


Fig. 13: Three cases of unified TCNN initialization [132], where σ^2 denotes the initial variance of each weight vertex. $\mathbf{G}_f$ denotes a forward procedure, and $\mathbf{G}_b$ denotes a backward procedure. (i) Standard convolution. The method in [132] degenerates to Xavier/Kaiming initialization on the standard convolution for the same weight variance formulation. (ii) Hyper Tucker-2 (HTK2) convolution. Tucker-2 (TK2) is a common TD that is utilized in ResNet as the bottleneck module [242]. HTK2 is formed by applying a hyperedge to the weight vertices of TK2. (iii) Odd convolution. The odd TD was originally proposed by [243]. The connections among the vertices are irregular, making weight initialization a complex problem. These three successful initialization cases can better demonstrate the potential adaptability of unified initialization to diverse TCNNs.

occurring in tensor contractions. Second, the diversity of tensor formats (e.g., CP decomposition, Tucker decomposition, Tensor Train) makes it challenging to develop a universally applicable initialization method that fits all tensorial layers. To address these limitations, Yu initialization [132] was proposed as a unified initialization paradigm. This method extends the principles of Xavier initialization while introducing adaptive mechanisms specifically designed for arbitrary Tensor-based Convolutional Neural Networks (TCNNs). The key innovation of Yu initialization lies in its systematic approach to handling tensor operations. Specifically, Pan *et al.* developed a two-step process: First, they extract a backbone graph (BG) from a tensorial convolution hypergraph [71], which captures the essential structure of tensor operations. Second, they encode an arbitrary TCNN into an adjacency matrix using this BG. Through this adjacency matrix representation, the method can directly calculate a suitable initial variance for any TCNN, taking into account its specific tensor structure and operations. We illustrate three representative cases of applying these unified initializations in Fig. 13. These examples demonstrate how the method adapts to different tensor formats and network architectures. Although Yu initialization was initially developed for TCNNs, its applicability extends far beyond this scope. The method has shown remarkable versatility and can be effectively applied to various neural network architectures.

5.2 Rank Selection and Search

Prior studies [76], [85], [234] focused on finding efficient TN formats (e.g., TTs and TRs) for compressing NNs and have achieved significant efficiency for their natural compact structures. However, despite these remarkable successes, efficient algorithms for adjusting or selecting suitable ranks for a TN are lacking since rank selection is an NP-hard problem [213]. As a result, many approaches [74], [76], [80], [84] can only set values for all ranks manually, which severely affects the resulting models' training procedures. Fortunately, the rank selection problem can still be optimized through heuristic strategies, such as Bayesian optimization [137], [138], reinforcement learning (RL) [135] and evolutionary algorithms (EAs) [134]. Here, we introduce some rank selection methods for TNNs.

DNNs utilize neural architecture search (NAS) [246] to search for the optimal network hyperparameters, achieving significant success. As ranks can be treated as architecture hyperparameters, NAS is applicable to searching for optimal tensorial layers with better rank settings. Following this idea, the progressive searching TR network (PSTRN) [134] employs NAS with an EA to select suitable ranks for a TR network (TRN). In detail, the PSTRN employs a heuristic hypothesis for searching: "when a shape-fixed TRN performs well, part or all of its rank elements are sensitive, and each of them tends to aggregate in a narrow region, which is called an interest region". Instructed by the interest region hypothesis, the PSTRN can reach the optimal point with a higher probability than a plain EA method. The PSTRN consists of an evolutionary phase and a progressive phase. During the evolutionary phase, this method validates the ranks in the search space on benchmarks and picks the rank that yields the best performance. Then, in the progressive phase, the PSTRN samples new ranks around the previously picked rank and inserts them into a new search space. After several rounds, the heuristic EA can find a high-performance solution. With such an efficient design, the PSTRN successfully achieves better performance than manual setting, which demonstrates that its hypothesis is practical.

In addition to NAS, some other efficient methods are also available for rank selection. Zhao *et al.* [136] inferred a CP rank by implementing a reduction process on a large rank value via a variational Bayesian optimization procedure. Hawkins and Zhang [138] extended this CP procedure [136] to TT-based TNNs and adopted the Stein variational gradient descent method, which combines the flexibility of the Markov chain Monte Carlo (MCMC) approach with the speed of variational Bayesian inference to construct a Bayesian optimization method. In pretrained networks, Kim *et al.* [141] and Gusak *et al.* [142] derived approximate ranks by employing Bayesian matrix factorization (BMF) [247] to unfolding weight tensors. Konstantin *et al.* [137] utilize a proxy-based Bayesian optimization approach to find the best combination of ranks for NN compression. Unlike Bayesian methods, Cheng *et al.* [135] treated the rank searching task as a game process whose search space was irregular, thus applying RL to find comparably suitable ranks for a trained CNN. However, this algorithm is TD-dependent, which indicates that its performance may be influenced by the selected TD method. Yin *et al.* [140] leveraged the alternating direction method of multipliers (ADMM) to gradually transfer the original weight to a low-rank representation (i.e., a TT). Solgi *et al.* [143] proposed a tensor reshaping optimization using genetic algorithms to improve tensor train (TT) decomposition compression efficiency by finding optimal tensor shapes, demonstrating significant improvements in image and neural network compression. Farnaz *et al.* [139] proposed an adaptive rank search framework for TR format in which TR ranks gradually increase in each iteration rather than being predetermined in advance.

5.3 Hardware Speedup

Accelerating the training and inference procedures of TNNs can benefit resource consumption and experimental adjustment, thereby achieving economic gains and green research. A direct and effective approach is to optimize the speed of tensor operations in TNNs to realize hardware acceleration. As inferring TT-format TNNs inevitably results in enormous quantities of redundant calculations, the TIE scheme [144] was proposed to accelerate TT layers by splitting the working SRAM into numerous groups with a well-designed data selection mechanism. Huang *et al.* [145] designed a parallel computation scheme with higher I/O bandwidth, improving the speed of tensor contractions. Later, they proposed an LTNN [145] to map TT-format TNNs into a 3D accelerator based on CMOS-RRAM, leading to significantly increased bandwidth via vertical I/O connections. As a result, they simultaneously attained high throughput and low power consumption for TNNs. Recently, Qu *et al.* [146] proposed a spatial 2D processing element (PE) array architecture and built a hardware TT engine consisting of off-chip DRAM. Kao *et al.* [147] proposed an energy-efficient hardware accelerator for CP convolution with a mixing method that combines the Walsh-Hadamard transform and the discrete cosine transform. ETTE [148] proposes a novel algorithm-hardware co-optimization framework for TT based TNN acceleration, featuring new tensor core construction, computation ordering mechanisms, and lookahead-style processing schemes, achieving significant improvements in computational efficiency, memory consumption, and data movement compared to existing solutions for various DNN architectures.

Many more fascinating methods have been developed for the acceleration of generic tensor operations, which are correlated with TNNs. For instance, Huang *et al.* [149] observed that the tensor matricization operation is usually resource-consuming since its DRAM access is built on a random reading address; thus, they proposed a tensor storage scheme with a sequential address design for better DRAM accessibility. Both T2s-tensor [150] and Tensaurus [151] mainly focus on designing general computation kernels for dense and sparse tensor data. Xie *et al.* [152] and Liang *et al.* [153] accelerated search procedures for obtaining an optimal sequence of tensor contractions. Xie *et al.* [152] solved the massive computational complexity problem of double-layer TN contraction in quantum analysis and mapped such a double-layer TN onto an intersected single-layer TN. Liang *et al.* [153] implemented multithread optimization to improve the parallelism of contractions. Fawzi *et al.* [154] also illustrated the potential of RL to build efficient universal tensor operations. In the future, it is expected that more general hardware acceleration schemes based on tensor operations will be developed to implement TNNs with smaller storage and time consumption levels.

Remark. The comments are divided into three parts. (1) To achieve training stability, it is possible to borrow ideas concerning identity transition maintenance to construct more stable initializations. In addition, it is also feasible to add adversarial examples to enhance network robustness. (2) Rank search is important for further improving the performance of TNNs. However, as it is an NP-hard problem, rank search has not been sufficiently explored. In the future, suitable ranks can be searched through the guidance of gradient sizes and EAs in searching for TNN architectures. (3) Last, research on hardware has derived some success in terms of speed acceleration and memory reduction. However, these methods are mostly ad hoc designs for specific TD formats, so they lack applicability to other TNN structures.

6 TNN Toolboxes

In 1973, Pereyra and Scherer [248], as pioneers in this field, developed a programming technique for basic tensor operations. Recently, with the development of modern computers, many more basic tensor operation toolboxes have been developed, and a series of powerful TNN toolboxes have also been proposed for both

network compression and quantum circuit simulation, which are the two main applications of TNNs. In this section, toolboxes for TNNs are presented in three categories according to their design purposes: (1) toolboxes for basic tensor operations, which contain important and fundamental operations (e.g., tensor contraction and permutation) in TNNs (Section 6.1); (2) toolboxes for network compression are high-level TNN architecture toolboxes based on other basic operation tools (Section 6.2); and (3) toolboxes for quantum circuit simulation are software packages for the quantum circuit simulation or quantum machine learning processes that use TNs from a quantum perspective (Section 6.3).

6.1 Toolboxes for Basic Tensor Operations

Toolboxes for basic tensor operations aim to implement some specific TD algorithms. Many basic tensor toolboxes based on different programming languages and backends have been designed for this purpose. For example, the online stochastic framework for TD (OSTD) [160] and Tensor Toolbox [157] were constructed for low-rank decomposition and implemented with MATLAB. Regarding Python-based toolboxes, TensorTools based on NumPy [249] implements CP only, while T3F [165] was explicitly designed for TT decomposition on TensorFlow [250]. Similarly, based on TensorFlow, TensorD [161] supports CP and Tucker decomposition. Tntorch [163] is a PyTorch-based library for tensor modeling in the CP, Tucker and TT formats. TorchMPS [122], TT-Toolbox [162] and Scikit-TT [167] are all powerful Python-based specific TT solvers that efficiently implement the DMRG algorithm. Tensorly is a powerful general TD library that supports many decomposition formats and various Python backends including CuPy, Pytorch, TensorFlow and MXNet [251]. TensorNetwork [166] is a powerful general-purpose TN library that supports a variety of Python backends, including JAX, TensorFlow, PyTorch and NumPy. HOTTBOX [158] provides comprehensive tools for tensor decomposition, multi-way analysis, and visualization of multi-dimensional data. In addition, some toolboxes based on C++ are also available. TenDeC++ [252] leverages a unique pointer technology called PointerDeformer in C++ to support the efficient computation of TD functions. ITensor [164] is an efficient and flexible C++ library for general TN calculations. Tensor4ML [253] provides a comprehensive overview of tensor decomposition models, algorithms, and optimization techniques, along with Python implementations and datasets, serving as a bridge between theoretical foundations and practical applications in machine learning and data science.

6.2 Toolboxes for Network Compression

Specific TNN toolboxes are used to assist with the development of tensorial layers. Although some general tensor toolboxes such as Tensorly [155] are powerful for TD processing and can use their TD operations to help initialize TNN modules to a certain extent, they still lack support for application programming interfaces (APIs) for building TNNs directly. Therefore, a TNN library (Tensorly-Torch) based on Tensorly was developed to build some tensor layers within any PyTorch network. Pan *et al.* also developed a powerful TNN library called TedNet [74]. TedNet can quickly set up TNN layers by directly calling the API. In addition, TedNet supports the construction of TCNNs and TRNNs in single lines of code.

6.3 Toolboxes for Quantum Simulation

A number of quantum circuit simulation toolboxes have been designed. For example, some TT toolboxes such as Scikit-TT and TorchMPS can partially simulate quantum circuits to some extent, although they were not specifically designed for quantum circuits simulation. In contrast, general TN toolboxes, e.g., TensorNetwork and ITensor, can simulate any quantum circuit. In addition, with optimized tensor contraction, TeD-Q [170], a TN-enhanced open-source software framework for quantum machine learning, enables the simulation of large quantum circuits. Furthermore, Yao [168], an extensible and efficient library for designing quantum algorithms, can provide support for dumping a quantum circuit into a TN. Although no practical implementations of quantum TNNs are available, these quantum circuit simulations are potentially useful for the simulation of quantum TNNs.

Remark. Despite the success of current toolboxes, some areas for improvement remain. (1) Existing basic tensor operation toolboxes are built using high-level software frameworks, limiting their ability to fully utilize the inherent capability of tensor computations. (2) Existing deep model implementation toolboxes for TNNs can only contain a limited number of predefined TNN structures and cannot allow users to design structures freely. (3) Existing quantum simulation toolboxes focus more on the simulation of quantum circuits using TNs and do not facilitate the processing of embedded quantum data via TNNs.

7 Discussion and Future Perspectives

7.1 Limitations and Critical Reflections of TNNs

While TNNs have shown promising advantages, several critical limitations need to be acknowledged. A primary concern is the computational complexity associated with tensor operations, particularly in high-dimensional spaces. Although TNNs theoretically offer efficient tensor decomposition, practical implementations often face significant computational bottlenecks, especially when scaling to large datasets or complex architectures. The optimization of TNNs presents unique challenges - the non-convex nature of tensor decomposition combined with neural network training can lead to convergence issues and local optima that are difficult to escape. Moreover, the robustness of TNNs to noise and perturbations in input data remains largely unexplored. The theoretical guarantees of TNs may not directly translate to practical stability in real-world applications. The interpretability of TNN models, while potentially better than traditional neural networks due to their structured nature, still presents significant challenges in extracting meaningful insights from learned representations. Additionally, the generalization ability of TNNs across different domains and tasks requires further investigation. Current success stories are often limited to specific applications, and the transfer of learned representations between different domains is not well understood. The field also lacks comprehensive empirical studies comparing TNNs with other state-of-the-art approaches across diverse benchmarks. These limitations highlight the need for more rigorous theoretical analysis and practical evaluations to fully understand the capabilities and constraints in real-world applications.

7.2 Connection to Low-rank Matrix Compression

While TNNs represent a significant advancement in neural network compression, it is important to understand their relationship with classical low-rank matrix compression methods. Although we focused on SVD as a representative example (as shown in recent

works like FWSVD-LLM [254], ASVD-LLM [255], and SVD-LLM [143]), there exists a rich family of matrix factorization techniques including the QR decomposition, LU decomposition, Non-negative Matrix Factorization (NMF), and CUR decomposition. Traditional matrix-based approaches compress neural networks by factorizing weight matrices into products of smaller matrices, exploiting low-rank properties to reduce parameters. TNNs extend this concept to higher-order tensors, offering several distinct advantages. First, unlike matrix methods that require flattening multi-dimensional data (potentially losing structural information), TNs preserve and leverage the natural multi-dimensional structure of the data and model parameters. Second, TNs provide more flexible decomposition formats (CP, Tucker, TT, etc.) that can be chosen based on specific data characteristics and computational requirements. Third, TN-based methods can often achieve better compression rates than matrix-based approaches when dealing with higher-order data, as they avoid the exponential scaling problem through their network structure. However, this connection to classical low-rank methods also highlights some shared challenges, such as rank selection and optimization stability, which remain active areas of research in both domains. Understanding this relationship helps explain both the theoretical foundations of TNNs and their practical advantages in neural network compression, while also suggesting potential directions for future improvements by combining insights from both approaches.

7.3 Acceleration based on hardware design

Although many TNNs have low calculation complexity levels in theory, realistic hardware deployments usually fall short of this objective due to their numerous permutation operations [74], [256] and the absence of sufficient parallelism [145]. The current hardware architectures are primarily designed for matrix operations, making them suboptimal for tensor operations that involve complex permutations and contractions. The frequent data movement between different memory hierarchies caused by permutation operations creates significant performance bottlenecks. This is particularly evident in operations like tensor transposition and reshaping, which require extensive data reorganization but contribute little to actual computation. While parallel computing frameworks like CUDA and OpenCL provide excellent support for matrix operations, their tensor operation capabilities are limited and often require multiple matrix operations to simulate a single tensor operation. This inefficiency is further compounded when dealing with higher-order tensors, where the overhead of decomposing tensor operations into multiple matrix operations becomes increasingly significant. Moreover, the current memory access patterns optimized for matrix operations may not be suitable for efficient tensor processing, leading to suboptimal cache utilization and increased memory latency. To address these challenges, several directions can be explored, including developing specialized tensor processing units (TPUs) [257], optimizing memory hierarchies for tensor-specific operations, and creating efficient tensor operation primitives at the hardware level. These solutions would need to consider both the computational aspects of tensor operations and the associated memory access patterns to achieve optimal performance.

7.4 Applications in quantum physics

In quantum physics applications involving large-scale tensors, TNNs offer unique advantages for efficiently handling complex quantum systems. A prime example is wave function simulation [258], where specifically designed TNNs can effectively process higher-order interactions that are computationally intractable for conventional methods. The potential of TNNs in quantum physics extends across multiple frontiers. In many-body quantum systems, TNNs excel at representing complex entanglement structures, providing a more efficient alternative to traditional approaches. Their tensor network structure naturally captures the quantum correlations and topological features inherent in these systems. For quantum state tomography, TNNs significantly reduce the computational complexity of reconstructing quantum states from experimental measurements, with their hierarchical structure allowing efficient compression of quantum state information while preserving essential physical properties. While simple neural networks have shown promise in tasks like free boson and fermion systems [259], they face significant scaling challenges. TNNs offer a natural solution through their inherent ability to handle high-dimensional tensors efficiently, preserving important physical properties like entanglement structure.

7.5 Implementations in quantum mechanics

The existing TNNs mainly adopt the mathematical forms of TNs and seldom consider the physical properties of the quantum systems described by these TNs [15], [131]. Several key aspects need to be addressed for implementing quantum TNNs. First, developing rigorous algorithms to map between simulated quantum TNNs and physical quantum systems remains a primary challenge. Second, methods to handle quantum noise and decoherence in physical implementations need to be established. Third, resource optimization techniques are essential to minimize quantum resources while maintaining computational advantages. Despite current hardware limitations, the theoretical foundation of quantum TNNs shows promise in inspiring more efficient classical TNN architectures and training methods. The deep connection between TNNs and quantum circuit structures suggests potential breakthroughs in both quantum and classical computing domains.

7.6 Potential usage of MERA

Multi-scale entanglement renormalization ansatz (MERA) [260], [261], [262] are a family of tree-like tensor networks that can be expressed in a hierarchical manner while maintaining significant computational benefits and tractability. MERA has demonstrated remarkable capabilities in capturing complex physical properties and intricate quantum correlations of strongly correlated ground states in quantum mechanics [260]. Its sophisticated hierarchical structure naturally supports multi-scale feature extraction and representation, making it particularly suitable for complex pattern recognition tasks and deep learning applications. The network's inherent ability to capture and preserve long-range correlations efficiently makes it especially ideal for tasks involving complex dependencies across different spatial and temporal scales. Furthermore, MERA's fundamental scale invariance properties can be especially beneficial for processing and analyzing data with multiple hierarchical scales, such as in image processing, signal analysis, and natural language understanding applications. The remarkable success of MERA in quantum many-body physics and quantum mechanics strongly suggests promising potential applications in designing more effective and computationally efficient classical machine learning algorithms and architectures.

7.7 Integration with Large Language Models.

The emergence of large language models (LLMs) presents exciting opportunities for integration with TNNs. TNNs could potentially enhance the efficiency and interpretability of attention mechanisms in transformer-based architectures, which are fundamental to modern LLMs. Their tensor structure could offer more compact representations of the complex relationships between tokens and provide more efficient ways to handle the quadratic complexity of attention mechanisms. Moreover, the hierarchical nature of some TN structures could be particularly valuable in modeling the nested relationships and multiple levels of abstraction present in natural language. The integration of TNNs with LLMs could also lead to more parameter-efficient architectures, reducing the computational resources required for training and inference while maintaining or even improving performance. Additionally, the theoretical foundations of TNs could provide new insights into the interpretability and theoretical understanding of large language models, potentially helping to bridge the gap between their empirical success and theoretical comprehension.

8 Conclusion

Tensor Networks (TNs) and Neural Networks (NNs) represent a compelling convergence of mathematical frameworks that, despite originating from distinct scientific disciplines, share profound theoretical connections. This survey systematically explores these connections and demonstrates how their integration creates powerful Tensorial Neural Networks (TNNs) with far-reaching implications. The theoretical foundation unifying these frameworks reveals that tensors provide a natural mathematical language for expressing the complex operations within neural networks. Through concepts like tensor convolution and convolutional tensors, we can formalize the operations in CNNs with greater mathematical rigor, leading to deeper understanding of their representational capabilities. This unified perspective enables cross-pollination of ideas between previously separate research communities, inspiring innovations in network architecture design and optimization techniques.

This theoretical convergence yields practical advances in sustainable AI through two complementary mechanisms. First, TNNs enable efficient data representation by naturally modeling higher-order interactions in multimodal, multiview and multitask scenarios, preserving structural information that would otherwise be lost in traditional flattening approaches. Second, tensor decomposition techniques provide remarkably compact model structures that substantially reduce parameter counts while maintaining or even enhancing performance, making deep learning more accessible in resource-constrained environments. Furthermore, TNNs create a natural bridge between classical and quantum computing paradigms. The mathematical structures of tensor networks align seamlessly with quantum system representations, making TNNs ideal for simulating quantum phenomena and developing quantum machine learning algorithms. This alignment positions TNNs as a promising framework for exploring quantum advantages in computational tasks while remaining implementable on classical hardware.

Looking forward, we believe TNNs will continue to evolve through advances in tensor-friendly hardware, novel tensor structures like MERA, and integration with emerging architectures such as large language models. By continuing this cross-disciplinary research, we can develop increasingly efficient, powerful, and interpretable AI systems that advance sustainable artificial intelligence while deepening our theoretical understanding of both neural networks and tensor mathematics.

References

[1] D. Cyganski and J. A. Orr, "Applications of tensor theory to object recognition and orientation determination," *IEEE Trans. PAMI*, no. 6, pp. 662–673, 1985.

[2] P. Koniusz, L. Wang, and A. Cherian, "Tensor representations for action recognition," *IEEE Trans. PAMI*, vol. 44, no. 2, pp. 648–665, 2021.

[3] J. Tang, X. Shu, G.-J. Qi, Z. Li, M. Wang, S. Yan, and R. Jain, "Tri-clustered tensor completion for social-aware image tag refinement," *IEEE Trans. PAMI*, vol. 39, no. 8, pp. 1662–1674, 2016.

[4] J. Tang, X. Shu, Z. Li, Y.-G. Jiang, and Q. Tian, "Social anchor-unit graph regularized tensor completion for large-scale image retagging," *IEEE Trans. PAMI*, vol. 41, no. 8, pp. 2027–2034, 2019.

[5] I. Davidson, S. Gilpin, O. Carmichael, and P. Walker, "Network discovery via constrained tensor analysis of fmri data," in *ACM SIGKDD*, 2013.

[6] E. Acar, Y. Levin-Schwartz, V. D. Calhoun, and T. Adali, "Tensor-based fusion of EEG and fMRI to understand neurological changes in schizophrenia," in *ISCAS*. IEEE, 2017.

[7] K. Keegan, T. Vishwanath, and Y. Xu, "A tensor SVD-based classification algorithm applied to fMRI data," *arXiv preprint arXiv:2111.00587*, 2021.

[8] I. Belyaeva, B. Gabrielson, Y.-P. Wang, T. W. Wilson, V. D. Calhoun, J. M. Stephen, and T. Adali, "Learning spatiotemporal brain dynamics in adolescents via multimodal meg and fmri data fusion using joint tensor/matrix decomposition," *IEEE Transactions on Biomedical Engineering*, 2024.

[9] Y. Liu, J. Li, J. L. Wisnowski, and R. M. Leahy, "Graph learning for cortical parcellation from tensor decompositions of resting-state fmri," *bioRxiv*, 2024.

[10] J. Biamonte and V. Bergholm, "Tensor networks in a nutshell," *arXiv preprint arXiv:1708.00006*, 2017.

[11] T. Huckle, K. Waldherr, and T. Schulte-Herbrüggen, "Computations in quantum tensor networks," *Linear Algebra and its Applications*, vol. 438, no. 2, pp. 750–781, 2013.

[12] M. S. Rudolph, J. Miller, D. Motlagh, J. Chen, A. Acharya, and A. Perdomo-Ortiz, "Synergistic pretraining of parametrized quantum circuits via tensor networks," *Nature Communications*, vol. 14, no. 1, p. 8367, 2023.

[13] X. Chen, Z. He, and L. Sun, "A Bayesian tensor decomposition approach for spatiotemporal traffic data imputation," *Transportation research part C: emerging technologies*, vol. 98, pp. 73–84, 2019.

[14] A. Cichocki, N. Lee, I. Oseledets, A.-H. Phan, Q. Zhao, D. P. Mandic *et al.*, "Tensor networks for dimensionality reduction and large-scale optimization: Part 1 low-rank tensor decompositions," *Foundations and Trends® in Machine Learning*, vol. 9, no. 4-5, pp. 249–429, 2016.

[15] A. Cichocki, A.-H. Phan, Q. Zhao, N. Lee, I. Oseledets, M. Sugiyama, D. P. Mandic *et al.*, "Tensor networks for dimensionality reduction and large-scale optimization: Part 2 applications and future perspectives," *Foundations and Trends® in Machine Learning*, vol. 9, no. 6, pp. 431–673, 2017.

[16] R. A. Harshman, "Foundations of the PARAFAC procedure: Models and conditions for an "explanatory" multi-modal factor analysis," *UCLA Working Papers in Phonetics*, vol. 16, pp. 1–84, 1970.

[17] J. D. Carroll and J.-J. Chang, "Analysis of individual differences in multidimensional scaling via an N-way generalization of "Eckart-Young" decomposition," *Psychometrika*, vol. 35, no. 3, pp. 283–319, 1970.

[18] H. A. Kiers, "Towards a standardized notation and terminology in multiway analysis," *Journal of Chemometrics: A Journal of the Chemometrics Society*, vol. 14, no. 3, pp. 105–122, 2000.

[19] L. R. Tucker, "Some mathematical notes on three-mode factor analysis," *Psychometrika*, vol. 31, no. 3, pp. 279–311, 1966.

[20] ——, "Implications of factor analysis of three-way matrices for measurement of change," *Problems in measuring change*, vol. 15, pp. 122–137, 1963.

[21] L. De Lathauwer, "Decompositions of a higher-order tensor in block terms—part I: Lemmas for partitioned matrices," *SIAM Journal on Matrix Analysis and Applications*, vol. 30, no. 3, pp. 1022–1032, 2008.

[22] I. V. Oseledets, "Tensor-train decomposition," *SIAM Journal on Scientific Computing*, vol. 33, no. 5, pp. 2295–2317, 2011.

[23] A. Cichocki, "Era of big data processing: A new approach via tensor networks and tensor decompositions," *arXiv preprint arXiv:1403.2048*, 2014.

[24] F. Verstraete, V. Murg, and J. I. Cirac, "Matrix product states, projected entangled pair states, and variational renormalization group methods for quantum spin systems," *Advances in Physics*, vol. 57, no. 2, pp. 143–224, 2008.

[25] Q. Zhao, G. Zhou, S. Xie, L. Zhang, and A. Cichocki, "Tensor ring decomposition," *arXiv preprint arXiv:1606.05535*, 2016.

[26] D. Kressner and C. Tobler, "htucker—a MATLAB toolbox for tensors in hierarchical Tucker format," *Mathicse, EPF Lausanne*, 2012.

[27] N. Schuch, M. M. Wolf, F. Verstraete, and J. I. Cirac, "Computational complexity of projected entangled pair states," *Physical Review Letters*, vol. 98, no. 14, p. 140506, 2007.

[28] A. Milsted and G. Vidal, "Geometric interpretation of the multi-scale entanglement renormalization ansatz," *arXiv preprint arXiv:1812.00529*, 2018.

[29] F. Pan and P. Zhang, "Simulation of quantum circuits using the big-batch tensor network method," *Physical Review Letters*, vol. 128, no. 3, p. 030501, 2022.

[30] Z. Zhang, G. I. Allen, H. Zhu, and D. Dunson, "Tensor network factorizations: Relationships between brain structural connectomes and traits," *Neuroimage*, vol. 197, pp. 330–343, 2019.

[31] A. Zare, A. Ozdemir, M. A. Iwen, and S. Aviyente, "Extension of pca to higher order data structures: An introduction to tensors, tensor decompositions, and tensor PCA," *Proceedings of the IEEE*, vol. 106, no. 8, pp. 1341–1358, 2018.

[32] J. Zhang, X. Li, P. Jing, J. Liu, and Y. Su, "Low-rank regularized heterogeneous tensor decomposition for subspace clustering," *IEEE Signal Processing Letters*, vol. 25, no. 3, pp. 333–337, 2017.

[33] A. Zadeh, M. Chen, S. Poria, E. Cambria, and L.-P. Morency, "Tensor fusion network for multimodal sentiment analysis," in *EMNLP*, 2017.

[34] Z. Liu, Y. Shen, V. B. Lakshminarasimhan, P. P. Liang, A. B. Zadeh, and L.-P. Morency, "Efficient low-rank multimodal fusion with modality-specific factors," in *ACL*, 2018.

[35] M. Hou, J. Tang, J. Zhang, W. Kong, and Q. Zhao, "Deep multimodal multilinear fusion with high-order polynomial pooling," *NeurIPS*, 2019.

[36] G. G. Chrysos, S. Moschoglou, G. Bouritsas, J. Deng, Y. Panagakis, and S. Zafeiriou, "Deep polynomial neural networks," *IEEE Trans. PAMI*, vol. 44, no. 8, pp. 4021–4034, 2021.

[37] A. Fukui, D. H. Park, D. Yang, A. Rohrbach, T. Darrell, and M. Rohrbach, "Multimodal compact bilinear pooling for visual question answering and visual grounding," in *EMNLP*, 2016.

[38] J.-H. Kim, K.-W. On, W. Lim, J. Kim, J.-W. Ha, and B.-T. Zhang, "Hadamard product for low-rank bilinear pooling," *arXiv preprint arXiv:1610.04325*, 2016.

[39] H. Ben-Younes, R. Cadene, M. Cord, and N. Thome, "Mutan: Multimodal Tucker fusion for visual question answering," in *ICCV*, 2017.

[40] T. Do, T.-T. Do, H. Tran, E. Tjiputra, and Q. D. Tran, "Compact trilinear interaction for visual question answering," in *ICCV*, 2019.

[41] L. He, B. Liu, G. Li, Y. Sheng, Y. Wang, and Z. Xu, "Knowledge base completion by variational Bayesian neural tensor decomposition," *Cognitive Computation*, pp. 1–10, 2018.

[42] T. Kwon, J. Ko, J. Jung, J.-G. Jang, and K. Shin, "Compact decomposition of irregular tensors for data compression: From sparse to dense to high-order tensors," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 1451–1462.

[43] T. Kwon, J. Ko, J. Jung, and K. Shin, "Tensorcodec: Compact lossy compression of tensors without strong data assumptions," in *2023 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2023, pp. 229–238.

[44] ——, "Neukron: Constant-size lossy compression of sparse reorderable matrices and tensors," in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 71–81.

[45] G. Novikov, A. Gneushev, A. Kadeishvili, and I. Oseledets, "Tensor-train point cloud compression and efficient approximate nearest-neighbor search," *arXiv preprint arXiv:2410.04462*, 2024.

[46] R. Ballester-Ripoll, P. Lindstrom, and R. Pajarola, "Tthresh: Tensor compression for multidimensional visual data," *IEEE transactions on visualization and computer graphics*, vol. 26, no. 9, pp. 2891–2903, 2019.

[47] J. Fan, "Multi-mode deep matrix and tensor factorization," in *international conference on learning representations*, 2021.

[48] D. Lee and K. Shin, "Robust factorization of real-world tensor streams with patterns, missing values, and outliers," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 840–851.

[49] H. Lamba, V. Nagarajan, K. Shin, and N. Shajarisales, "Incorporating side information in tensor completion," in *Proceedings of the 25th International Conference Companion on World Wide Web*, 2016, pp. 65–66.

[50] M. Wang, D. Zeng, Z. Xu, R. Guo, and X. Zhao, "Federated knowledge graph completion via latent embedding sharing and tensor factorization," in *2023 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2023, pp. 1361–1366.

[51] Y. Yang and T. Hospedales, "Deep multi-task representation learning: A tensor factorisation approach," in *ICLR*, 2017.

[52] M. Wang, Z. Su, X. Luo, Y. Pan, S. Zheng, and Z. Xu, "Concatenated tensor networks for deep multi-task learning," in *ICONIP*, 2020.

[53] M. Duan, K. Li, K. Li, and Q. Tian, "A novel multi-task tensor correlation neural network for facial attribute prediction," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 12, no. 1, pp. 1–22, 2020.

[54] Z. Zhang, Y. Xie, W. Zhang, Y. Tang, and Q. Tian, "Tensor multi-task learning for person re-identification," *IEEE Transactions on Image Processing*, vol. 29, pp. 2463–2477, 2019.

[55] Y. Zhang, Y. Zhang, and W. Wang, "Multi-task learning via generalized tensor trace norm," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 2254–2262.

[56] X. Jin, J. Tang, X. Kong, Y. Peng, J. Cao, Q. Zhao, and W. Kong, "Ctnn: A convolutional tensor-train neural network for multi-task brainprint recognition," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 29, pp. 103–112, 2020.

[57] X. Li, K. S. Candan, and M. L. Sapino, "M2td: multi-task tensor decomposition for sparse ensemble simulations," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 1144–1155.

[58] Y. Zhang, P. Yang, and V. Lanfranchi, "Tensor multi-task learning for predicting alzheimer's disease progression using mri data with spatio-temporal similarity measurement," in *2021 IEEE 19th International Conference on Industrial Informatics (INDIN)*. IEEE, 2021, pp. 1–8.

[59] Y. Garg, N. Yismaw, R. Hyder, A. Prater-Bennette, and M. S. Asif, "Factorized tensor networks for multi-task and multi-domain learning," *arXiv preprint arXiv:2310.06124*, 2023.

[60] Y. Ren, J. Lou, L. Xiong, J. C. Ho, X. Jiang, and S. V. Bhavani, "Multipar: Supervised irregular tensor factorization with multi-task learning for computational phenotyping," in *Machine Learning for Health (ML4H)*. PMLR, 2023, pp. 498–511.

[61] H. Liu, M. Liu, J. Wang, X. Xie, and L. Yang, "Non-intrusive speech quality assessment with multi-task learning based on tensor network," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 851–855.

[62] J. Xu, J. Zhou, P.-N. Tan, X. Liu, and L. Luo, "Wisdom: Weighted incremental spatio-temporal multi-task learning via tensor decomposition," in *2016 IEEE International Conference on Big Data (Big Data)*. IEEE, 2016, pp. 522–531.

[63] R. Wang, J. Zhu, S. Wang, T. Wang, J. Huang, and X. Zhu, "Multimodal emotion recognition using tensor decomposition fusion and self-supervised multi-tasking," *International Journal of Multimedia Information Retrieval*, vol. 13, no. 4, p. 39, 2024.

[64] E. Stoudenmire and D. J. Schwab, "Supervised learning with tensor networks," *NeurIPS*, 2016.

[65] G. Gan, P. Zhang, S. Li, X. Lu, and B. Wang, "Morphte: Injecting morphology in tensorized embeddings," in *NeurIPS*, 2022.

[66] Q. Li, B. Wang, and M. Melucci, "CNM: An interpretable complex-valued network for matching," in *NAACL*, 2019.

[67] P. Zhang, Z. Su, L. Zhang, B. Wang, and D. Song, "A quantum many-body wave function inspired language modeling approach," in *CIKM*, 2018.

[68] J. Miller, G. Rabusseau, and J. Terilla, "Tensor networks for probabilistic sequence modeling," in *AISTATS*, 2021.

[69] E. L. Denton, W. Zaremba, J. Bruna, Y. LeCun, and R. Fergus, "Exploiting linear structure within convolutional networks for efficient evaluation," in *NeurIPS*, 2014, pp. 1269–1277.

[70] V. Lebedev, Y. Ganin, M. Rakhuba, I. Oseledets, and V. Lempitsky, "Speeding-up convolutional neural networks using fine-tuned cp-decomposition," in *ICLR*, 2015.

[71] K. Hayashi, T. Yamaguchi, Y. Sugawara, and S. Maeda, "Exploring unexplored tensor network decompositions for convolutional neural networks," in *NeurIPS*, 2019.

[72] A.-H. Phan, K. Sobolev, K. Sozykin, D. Ermilov, J. Gusak, P. Tichavskỳ, V. Glukhov, I. Oseledets, and A. Cichocki, "Stable low-rank tensor decomposition for compression of convolutional neural network," in *ECCV*. Springer, 2020, pp. 522–539.

[73] A. Nekooei and S. Safari, "Compression of deep neural networks based on quantized tensor decomposition to implement on reconfigurable hardware platforms," *Neural Networks*, vol. 150, pp. 350–363, 2022.

[74] Y. Pan, M. Wang, and Z. Xu, "Tednet: A Pytorch toolkit for tensor decomposition networks," *Neurocomputing*, vol. 469, pp. 234–238, 2022.

[75] Y. Liu and M. K. Ng, "Deep neural network compression by Tucker decomposition with nonlinear response," *Knowledge-Based Systems*, 2022.

[76] J. Ye, G. Li, D. Chen, H. Yang, S. Zhe, and Z. Xu, "Block-term tensor neural networks." *Neural Networks: the Official Journal of the International Neural Network Society*, vol. 130, pp. 11–21, 2020.

[77] T. Garipov, D. Podoprikhin, A. Novikov, and D. Vetrov, "Ultimate tensorization: compressing convolutional and fc layers alike," *arXiv preprint arXiv:1611.03214*, 2016.

[78] D. Liu, L. T. Yang, P. Wang, R. Zhao, and Q. Zhang, "Tt-tsvd: A multi-modal tensor train decomposition with its application in convolutional neural networks for smart healthcare," *TOMM*, vol. 18, no. 1s, pp. 1–17, 2022.

[79] J. Qi, C.-H. H. Yang, P.-Y. Chen, and J. Tejedor, "Exploiting low-rank tensor-train deep neural networks based on riemannian gradient descent with illustrations of speech processing," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 633–642, 2023.

[80] W. Wang, Y. Sun, B. Eriksson, W. Wang, and V. Aggarwal, "Wide compression: Tensor ring nets," in *CVPR*, 2018, pp. 9329–9338.

[81] J. Kossaifi, A. Bulat, G. Tzimiropoulos, and M. Pantic, "T-net: Parametrizing fully convolutional nets with a single high-order tensor," in *CVPR*, 2019.

[82] K. Xie, C. Liu, X. Wang, X. Li, G. Xie, J. Wen, and K. Li, "Neural network compression based on tensor ring decomposition," *IEEE Transactions on Neural Networks and Learning Systems*, 2024.

[83] J. Kossaifi, A. Toisoul, A. Bulat, Y. Panagakis, T. M. Hospedales, and M. Pantic, "Factorized higher-order cnns with an application to spatio-temporal emotion estimation," in *CVPR*, 2020.

[84] Y. Yang, D. Krompass, and V. Tresp, "Tensor-Train recurrent neural networks for video classification," in *ICML*, 2017.

[85] Y. Pan, J. Xu, M. Wang, J. Ye, F. Wang, K. Bai, and Z. Xu, "Compressing recurrent neural networks with tensor ring for action recognition," in *AAAI*, 2019.

[86] J. Ye, L. Wang, G. Li, D. Chen, S. Zhe, X. Chu, and Z. Xu, "Learning compact recurrent neural networks with block-term tensor decomposition," in *CVPR*, 2018.

[87] A. Tjandra, S. Sakti, and S. Nakamura, "Recurrent neural network compression based on low-rank tensor representation," *IEICE Trans. Inf. Syst.*, vol. 103-D, no. 2, pp. 435–449, 2020.

[88] M. Yin, S. Liao, X. Liu, X. Wang, and B. Yuan, "Towards extremely compact rnns for video recognition with fully decomposed hierarchical tucker structure," in *CVPR*, 2021.

[89] B. Wu, D. Wang, G. Zhao, L. Deng, and G. Li, "Hybrid tensor decomposition in neural network compression," *Neural Networks*, vol. 132, pp. 309–320, 2020.

[90] J. Su, W. Byeon, J. Kossaifi, F. Huang, J. Kautz, and A. Anandkumar, "Convolutional tensor-train LSTM for spatio-temporal learning," in *NeurIPS*, 2020.

[91] J. Kossaifi, Z. C. Lipton, A. Kolbeinsson, A. Khanna, T. Furlanello, and A. Anandkumar, "Tensor regression networks," *J. Mach. Learn. Res.*, vol. 21, no. 123, pp. 1–21, 2020.

[92] J. Tangpanitanon, C. Mangkang, P. Bhadola, Y. Minato, D. G. Angelakis, and T. Chotibut, "Explainable natural language processing with matrix product states," *New Journal of Physics*, vol. 24, no. 5, p. 053032, 2022.

[93] P. Liu, Z. Gao, W. X. Zhao, Z. Xie, Z. Lu, and J. Wen, "Enabling lightweight fine-tuning for pre-trained language model compression based on matrix product operators," in *ACL/IJCNLP*, 2021.

[94] L. Sunzhu, Z. Peng, G. Guobing, L. Xiuqing, W. Benyou, W. Junqiu, and J. Xin, "Hypoformer: Hybrid decomposition transformer for edge-friendly neural machine translation," *EMNLP*, 2022.

[95] B. Wang, Y. Ren, L. Shang, X. Jiang, and Q. Liu, "Exploring extreme parameter compression for pre-trained language models," in *ICLR*, 2022.

[96] J. Tang, K. Li, M. Hou, X. Jin, W. Kong, Y. Ding, and Q. Zhao, "Mmt: Multi-way multi-modal transformer for multimodal learning," in *IJCAI*, 2022.

[97] J. Zhao, F. Zhuo, Q. Sun, Q. Li, Y. Hua, and J. Zhao, "Tensor compressed transformer network for traffic flow forecasting," *Available at SSRN 4502093*.

[98] Y. Zhang, Y. Liu, H. Yuan, Z. Qin, Y. Yuan, Q. Gu, and A. C.-C. Yao, "Tensor product attention is all you need," 2025.

[99] X. Liu, J. Su, and F. Huang, "Tuformer: Data-driven design of expressive transformer by Tucker tensor representation," in *ICLR*, 2022.

[100] M. A. O. Vasilescu, "Causal deep learning: Causal capsules and tensor transformers," *arXiv preprint arXiv:2301.00314*, 2023.

[101] C. Hua, G. Rabusseau, and J. Tang, "High-order pooling for graph neural networks with tensor decomposition," *NeurIPS*, 2022.

[102] P. Baghershahi, R. Hosseini, and H. Moradi, "Efficient relation-aware neighborhood aggregation in graph neural networks via tensor decomposition," *arXiv preprint arXiv:2212.05581*, 2022.

[103] C. Yin, D. Zheng, I. Nisa, C. Faloutsos, G. Karypis, and R. Vuduc, "Nimble gnn embedding with tensor-train decomposition," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 2327–2335.

[104] X. Zhao, Q. Dai, J. Wu, H. Peng, M. Liu, X. Bai, J. Tan, S. Wang, and P. Yu, "Multi-view tensor graph neural networks through reinforced aggregation," *TKDE*, 2022.

[105] M. Wang, Y. Zhen, Y. Pan, Y. Zhao, C. Zhuang, Z. Xu, R. Guo, and X. Zhao, "Tensorized hypergraph neural networks," in *Proceedings of the 2024 SIAM International Conference on Data Mining (SDM)*. SIAM, 2024, pp. 127–135.

[106] C. Jia, B. Wu, and X.-P. Zhang, "Dynamic spatiotemporal graph neural network with tensor network," *arXiv preprint arXiv:2003.08729*, 2020.

[107] M. Xu, Y. L. Xu, and D. P. Mandic, "Tensorgpt: Efficient compression of the embedding layer in llms based on the tensor-train decomposition," *arXiv preprint arXiv:2307.00526*, 2023.

[108] A. Tomut, S. S. Jahromi, A. Sarkar, U. Kurt, S. Singh, F. Ishtiaq, C. Muñoz, P. S. Bajaj, A. Elborady, G. del Bimbo *et al.*, "Compactifai: extreme compression of large language models using quantum-inspired tensor networks," *arXiv preprint arXiv:2401.14109*, 2024.

[109] A. Basharin, A. Chertkov, and I. Oseledets, "Faster language models with better multi-token prediction using tensor decomposition," *arXiv preprint arXiv:2410.17765*, 2024.

[110] V. Chekalina, G. Novikov, J. Gusak, I. Oseledets, and A. Panchenko, "Efficient gpt model pre-training using tensor train matrix representation," *arXiv preprint arXiv:2306.02697*, 2023.

[111] V. Abronin, A. Naumov, D. Mazur, D. Bystrov, K. Tsarova, A. Melnikov, S. Dolgov, R. Brasher, and M. Perelshein, "Tqcompressor: improving tensor decomposition methods in neural networks via permutations," in *2024 IEEE 7th International Conference on Multimedia Information Processing and Retrieval (MIPR)*. IEEE, 2024, pp. 503–506.

[112] A. Anjum, M. E. Eren, I. Boureima, B. Alexandrov, and M. Bhattarai, "Tensor train low-rank approximation (tt-lora): Democratizing ai with accelerated llms," *arXiv preprint arXiv:2408.01008*, 2024.

[113] X. Chen, J. Liu, Y. Wang, P. Wang, M. Brand, G. Wang, and T. Koike-Akino, "Superlora: Parameter-efficient unified adaptation for large vision models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 8050–8055.

[114] T. Koike-Akino, F. Tonin, Y. Wu, L. N. Candogan, and V. Cevher, "Quantum-peft: Ultra parameter-efficient fine-tuning," in *Workshop on Efficient Systems for Foundation Models II@ ICML2024*.

[115] G. He, W. Cheng, H. Zhu, and G. Yu, "Lora-pt: Low-rank adapting unetr for hippocampus segmentation using principal tensor singular values and vectors," *arXiv preprint arXiv:2407.11292*, 2024.

[116] C. Si, X. Wang, X. Yang, Z. Xu, Q. Li, J. Dai, Y. Qiao, X. Yang, and W. Shen, "Flora: Low-rank core space for n-dimension," *arXiv preprint arXiv:2405.14739*, 2024.

[117] D. Bershatsky, D. Cherniuk, T. Daulbaev, A. Mikhalev, and I. Oseledets, "Lotr: Low tensor rank weight adaptation," *arXiv preprint arXiv:2402.01376*, 2024.

[118] M. Xu, S. Sharmin, and D. P. Mandic, "Geometry is all you need: A unified taxonomy of matrix and tensor factorization for compression of generative language models," *arXiv preprint arXiv:2410.03040*, 2024.

[119] Z. Chen, R. Dangovski, C. Loh, O. Dugan, D. Luo, and M. Soljačić, "Quanta: Efficient high-rank fine-tuning of llms with quantum-informed tensor adaptation," *arXiv preprint arXiv:2406.00132*, 2024.

[120] S. Jie and Z.-H. Deng, "Fact: Factor-tuning for lightweight adaptation on vision transformer," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, no. 1, 2023, pp. 1060–1068.

[121] X. Hu, X. Cheng, P. Liu, W. Liu, J. Luan, B. Wang, and Y. Liu, "Dota: Weight-decomposed tensor adaptation for large language models," *arXiv preprint arXiv:2412.20891*, 2024.

[122] J. Miller, "TorchMPS," 2019. [Online]. Available: https://github.com/jemisjoky/torchmps

[123] M. Born, "Quantenmechanik der stoßvorgänge," *Zeitschrift für Physik*, vol. 38, no. 11, pp. 803–827, 1926.

[124] N. Cohen, O. Sharir, and A. Shashua, "On the expressive power of deep learning: A tensor analysis," in *COLT*, 2016.

[125] Y. Levine, O. Sharir, N. Cohen, and A. Shashua, "Quantum entanglement in deep learning architectures," *Physical Review Letters*, vol. 122, no. 6, p. 065301, 2019.

[126] L. Zhang, P. Zhang, X. Ma, S. Gu, Z. Su, and D. Song, "A generalized language model in tensor space," in *AAAI*, 2019.

[127] Z. Chen, L. Newhouse, E. Chen, D. Luo, and M. Soljacic, "Antn: Bridging autoregressive neural networks and tensor networks for quantum many-body simulation," *Advances in Neural Information Processing Systems*, vol. 36, pp. 450–476, 2023.

[128] Y. Qing, K. Li, P.-F. Zhou, and S.-J. Ran, "Compressing neural network by tensor network with exponentially fewer variational parameters," *arXiv preprint arXiv:2305.06058*, 2023.

[129] Z. Su, Y. Zhou, F. Mo, and J. G. Simonsen, "Language modeling using tensor trains," *arXiv preprint arXiv:2405.04590*, 2024.

[130] W.-Y. Liu, S.-J. Du, R. Peng, J. Gray, and G. K. Chan, "Tensor network computations that capture strict variationality, volume law behavior, and the efficient representation of neural network states," *arXiv preprint arXiv:2405.03797*, 2024.

[131] Y. Panagakis, J. Kossaifi, G. G. Chrysos, J. Oldfield, M. A. Nicolaou, A. Anandkumar, and S. Zafeiriou, "Tensor methods in computer vision and deep learning," *Proc. IEEE*, vol. 109, no. 5, pp. 863–890, 2021.

[132] Y. Pan, Z. Su, A. Liu, J. Wang, N. Li, and Z. Xu, "A unified weight initialization paradigm for tensorial convolutional neural networks," in *ICML*, 2022.

[133] Y. Pan, Y. Yuan, Y. Yin, Z. Xu, L. Shang, X. Jiang, and Q. Liu, "Reusing pretrained models by multi-linear operators for efficient training," *Advances in Neural Information Processing Systems*, vol. 36, pp. 3248–3262, 2023.

[134] N. Li, Y. Pan, Y. Chen, Z. Ding, D. Zhao, and Z. Xu, "Heuristic rank selection with progressively searching tensor ring network," *Complex & Intelligent Systems*, pp. 1–15, 2021.

[135] Z. Cheng, B. Li, Y. Fan, and Y. Bao, "A novel rank selection scheme in tensor ring decomposition based on reinforcement learning for deep neural networks," in *ICASSP*, 2020.

[136] Q. Zhao, L. Zhang, and A. Cichocki, "Bayesian CP factorization of incomplete tensors with automatic rank determination," *IEEE Trans. PAMI*, vol. 37, no. 9, pp. 1751–1763, 2015.

[137] K. Sobolev, D. Ermilov, A.-H. Phan, and A. Cichocki, "Pars: Proxy-based automatic rank selection for neural network compression via low-rank weight approximation," *Mathematics*, vol. 10, no. 20, p. 3801, 2022.

[138] C. Hawkins and Z. Zhang, "Bayesian tensorized neural networks with automatic rank selection," *Neurocomputing*, vol. 453, pp. 172–180, 2021.

[139] F. Sedighin, A. Cichocki, and A.-H. Phan, "Adaptive rank selection for tensor ring decomposition," *IEEE Journal of Selected Topics in Signal Processing*, vol. 15, no. 3, pp. 454–463, 2021.

[140] M. Yin, Y. Sui, S. Liao, and B. Yuan, "Towards efficient tensor decomposition-based DNN model compression with optimization framework," in *CVPR*, 2021.

[141] Y. Kim, E. Park, S. Yoo, T. Choi, L. Yang, and D. Shin, "Compression of deep convolutional neural networks for fast and low power mobile applications," in *ICLR*, 2016.

[142] J. Gusak, M. Kholyavchenko, E. Ponomarev, L. Markeeva, P. Blagoveschensky, A. Cichocki, and I. V. Oseledets, "Automated multi-stage compression of neural networks," in *ICCV Workshops*, 2019.

[143] X. Wang, Y. Zheng, Z. Wan, and M. Zhang, "Svd-llm: Truncation-aware singular value decomposition for large language model compression," *arXiv preprint arXiv:2403.07378*, 2024.

[144] C. Deng, F. Sun, X. Qian, J. Lin, Z. Wang, and B. Yuan, "TIE: energy-efficient tensor train-based inference engine for deep neural network," in *ISCA*, 2019.

[145] H. Huang, L. Ni, and H. Yu, "LTNN: An energy-efficient machine learning accelerator on 3d cmos-rram for layer-wise tensorized neural network," in *SOCC*, 2017.

[146] Z. Qu, L. Deng, B. Wang, H. Chen, J. Lin, L. Liang, G. Li, Z. Zhang, and Y. Xie, "Hardware-enabled efficient data processing with tensor-train decomposition," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 41, no. 2, pp. 372–385, 2022.

[147] C. Kao, Y. Hsieh, C. Chen, and C. Yang, "Hardware acceleration in large-scale tensor decomposition for neural network compression," in *MWSCAS*, 2022.

[148] Y. Gong, M. Yin, L. Huang, J. Xiao, Y. Sui, C. Deng, and B. Yuan, "Ette: Efficient tensor-train-based computing engine for deep neural networks," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–13.

[149] L. Huang, C. Deng, S. Ibrahim, X. Fu, and B. Yuan, "VLSI hardware architecture of stochastic low-rank tensor decomposition," in *ACSCC*, 2021.

[150] N. Srivastava, H. Rong, P. Barua, G. Feng, H. Cao, Z. Zhang, D. Albonesi, V. Sarkar, W. Chen, P. Petersen *et al.*, "T2S-Tensor: Productively generating high-performance spatial hardware for dense tensor computations," in *FCCM*, 2019.

[151] N. Srivastava, H. Jin, S. Smith, H. Rong, D. Albonesi, and Z. Zhang, "Tensaurus: A versatile accelerator for mixed sparse-dense tensor computations," in *HPCA*, 2020.

[152] Z. Xie, H. Liao, R. Huang, H. Xie, J. Chen, Z. Liu, and T. Xiang, "Optimized contraction scheme for tensor-network states," *Physical Review B*, vol. 96, no. 4, p. 045128, 2017.

[153] L. Liang, J. Xu, L. Deng, M. Yan, X. Hu, Z. Zhang, G. Li, and Y. Xie, "Fast search of the optimal contraction sequence in tensor networks," *IEEE Journal of Selected Topics in Signal Processing*, vol. 15, no. 3, pp. 574–586, 2021.

[154] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R Ruiz, J. Schrittwieser, G. Swirszcz *et al.*, "Discovering faster matrix multiplication algorithms with reinforcement learning," *Nature*, vol. 610, no. 7930, pp. 47–53, 2022.

[155] J. Kossaifi, Y. Panagakis, A. Anandkumar, and M. Pantic, "Tensorly: Tensor learning in python," *J. Mach. Learn. Res.*, vol. 20, pp. 26:1–26:6, 2019.

[156] A. H. Williams, T. H. Kim, F. Wang, S. Vyas, S. I. Ryu, K. V. Shenoy, M. Schnitzer, T. G. Kolda, and S. Ganguli, "Unsupervised discovery of demixed, low-dimensional neural dynamics across multiple timescales through tensor component analysis," *Neuron*, vol. 98, no. 6, pp. 1099–1115, 2018.

[157] T. G. Kolda and B. W. Bader, "Matlab tensor toolbox," Sandia National Laboratories (SNL), Tech. Rep., 2006.

[158] I. Kisil, G. G. Calvi, B. S. Dees, and D. P. Mandic, "Hottbox: Higher order tensor toolbox," *arXiv preprint arXiv:2111.15662*, 2021.

[159] J. Huang, L. Kong, X. Liu, W. Qu, and G. Chen, "A C++ library for tensor decomposition," in *IPCCC*, 2019.

[160] A. Sobral, S. Javed, S. K. Jung, T. Bouwmans, and E. Zahzah, "Online stochastic tensor decomposition for background subtraction in multispectral video sequences," in *ICCV Workshops*, 2015.

[161] L. Hao, S. Liang, J. Ye, and Z. Xu, "TensorD: A tensor decomposition library in tensorflow," *Neurocomputing*, vol. 318, pp. 196–200, 2018.

[162] I. Oseledets, S. Dolgov, V. Kazeev, D. Savostyanov, O. Lebedeva, P. Zhlobich, T. Mach, and L. Song, "TT-toolbox," 2016.

[163] R. Ballester-Ripoll, "tntorch - tensor network learning with PyTorch," 2018. [Online]. Available: https://github.com/rballester/tntorch

[164] M. Fishman, S. R. White, and E. M. Stoudenmire, "The ITensor software library for tensor network calculations," *arXiv preprint arXiv:2007.14822*, 2020.

[165] A. Novikov, P. Izmailov, V. Khrulkov, M. Figurnov, and I. V. Oseledets, "Tensor train decomposition on tensorflow (T3F)," *J. Mach. Learn. Res.*, vol. 21, pp. 30:1–30:7, 2020.

[166] C. Roberts, A. Milsted, M. Ganahl, A. Zalcman, B. Fontaine, Y. Zou, J. Hidary, G. Vidal, and S. Leichenauer, "Tensornetwork: A library for physics and machine learning," *arXiv preprint arXiv:1905.01330, 2019*, 2019.

[167] P. Gel, S. Klus, M. Scherer, and F. Nske, "Scikit-TT tensor train toolbox," 2018. [Online]. Available: https://github.com/PGelss/scikit$/_$tt

[168] X.-Z. Luo, J.-G. Liu, P. Zhang, and L. Wang, "Yao. jl: Extensible, efficient framework for quantum algorithm design," *Quantum*, vol. 4, p. 341, 2020.

[169] D. Kartsaklis, I. Fan, R. Yeung, A. Pearson, R. Lorenz, A. Toumi, G. de Felice, K. Meichanetzidis, S. Clark, and B. Coecke, "lambeq: An efficient high-level python library for quantum nlp," *arXiv preprint arXiv:2110.04236*, 2021.

[170] J. E. Academy, "Tensor-network enhanced distributed quantum," 2022. [Online]. Available: https://github.com/JDEA-Quantum-Lab/TeD-Q

[171] J. J. Hopfield, "Neural networks and physical systems with emergent collective computational abilities," *Proceedings of the national academy of sciences*, vol. 79, no. 8, pp. 2554–2558, 1982.

[172] D. E. Rumelhart, G. E. Hinton, R. J. Williams *et al.*, "Learning representations by back-propagating errors," *Cognitive Modeling*, vol. 5, no. 3, p. 1, 1988.

[173] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural Networks*, vol. 61, pp. 85–117, 2015.

[174] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, p. 436, 2015.

[175] G. E. Hinton, "A practical guide to training restricted Boltzmann machines," in *Neural Networks: Tricks of the Trade*. Springer, 2012, pp. 599–619.

[176] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *CVPR*, 2017.

[177] P. Zhou, W. Shi, J. Tian, Z. Qi, B. Li, H. Hao, and B. Xu, "Attention-based bidirectional long short-term memory networks for relation classification," in *ACL*, 2016.

[178] R. Dey and F. M. Salem, "Gate-variants of gated recurrent unit (GRU) neural networks," in *MWSCAS*, 2017.

[179] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeurIPS*, 2017.

[180] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *NAACL-HLT*, 2019.

[181] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *ICLR*, 2020.

[182] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Huggingface's transformers: State-of-the-art natural language processing," *arXiv preprint arXiv:1910.03771*, 2019.

[183] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein *et al.*, "ImageNet large scale visual recognition challenge," *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015.

[184] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *NeurIPS*, 2012.

[185] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[186] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *CVPR*, 2015.

[187] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *CVPR*, 2016.

[188] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko *et al.*, "Highly accurate protein structure prediction with AlphaFold," *Nature*, vol. 596, no. 7873, pp. 583–589, 2021.

[189] M. van Breugel, I. Rosa e Silva, and A. Andreeva, "Structural validation and assessment of AlphaFold2 predictions for centrosomal and centriolar proteins and their complexes," *Communications Biology*, vol. 5, no. 1, pp. 1–10, 2022.

[190] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[191] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023.

[192] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[193] Anthropic, "Model card: Claude 3," Anthropic, Tech. Rep., 2024. [Online]. Available: https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf

[194] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[195] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[196] T. GLM, A. Zeng, B. Xu, B. Wang, C. Zhang, D. Yin, D. Zhang, D. Rojas, G. Feng, H. Zhao *et al.*, "Chatglm: A family of large language models from glm-130b to glm-4 all tools," *arXiv preprint arXiv:2406.12793*, 2024.

[197] A. S. Dhanjal and W. Singh, "A comprehensive survey on automatic speech recognition using neural networks," *Multimedia Tools and Applications*, vol. 83, no. 8, pp. 23 367–23 412, 2024.

[198] P. Parhami, M. Fateh, M. Rezvani, and H. Alinejad-Rokny, "A comparison of deep neural network models for cluster cancer patients through somatic point mutations," *Journal of Ambient Intelligence and Humanized Computing*, vol. 14, no. 8, pp. 10 883–10 898, 2023.

[199] G. Ahdritz, N. Bouatta, S. Kadyan, L. Jarosch, D. Berenberg, I. Fisk, A. Watkins, S. Ra, R. Bonneau, and M. AlQuraishi, "Openproteinset: Training data for structural biology at scale," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[200] H. Chen, O. Engkvist, Y. Wang, M. Olivecrona, and T. Blaschke, "The rise of deep learning in drug discovery," *Drug discovery today*, vol. 23, no. 6, pp. 1241–1250, 2018.

[201] Y. Zhou, E. Lentz, H. Michelson, C. Kim, and K. Baylis, "Machine learning for food security: Principles for transparency and usability," *Applied Economic Perspectives and Policy*, vol. 44, no. 2, pp. 893–910, 2022.

[202] M. Mondelli and A. Montanari, "On the connection between learning two-layer neural networks and tensor decomposition," in *The 22nd International Conference on Artificial Intelligence and Statistics*. PMLR, 2019, pp. 1051–1060.

[203] J. Chen, S. Cheng, H. Xie, L. Wang, and T. Xiang, "Equivalence of restricted Boltzmann machines and tensor network states," *Physical Review B*, vol. 97, no. 8, p. 085104, 2018.

[204] S. R. Clark, "Unifying neural-network quantum states and correlator product states via tensor networks," *Journal of Physics A: Mathematical and Theoretical*, vol. 51, no. 13, p. 135301, 2018.

[205] S. Rendle, "Factorization machines," in *ICDM*, 2010.

[206] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick, and D. Parikh, "Vqa: Visual question answering," in *ICCV*, 2015.

[207] Z. Xu, F. Yan, and Y. Qi, "Infinite Tucker decomposition: nonparametric Bayesian models for multiway data analysis," in *ICML*, 2012.

[208] S. Zhe, Y. Qi, Y. Park, Z. Xu, I. Molloy, and S. Chari, "Dintucker: Scaling up gaussian process models on large multidimensional arrays," in *AAAI*, 2016.

[209] R. Penrose, "Applications of negative dimensional tensors," *Combinatorial Mathematics and its Applications*, vol. 1, pp. 221–244, 1971.

[210] T. G. Kolda and B. W. Bader, "Tensor decompositions and applications," *SIAM Review*, vol. 51, no. 3, pp. 455–500, 2009.

[211] N. D. Sidiropoulos, L. De Lathauwer, X. Fu, K. Huang, E. E. Papalexakis, and C. Faloutsos, "Tensor decomposition for signal processing and machine learning," *IEEE Transactions on Signal Processing*, vol. 65, no. 13, pp. 3551–3582, 2017.

[212] U. Schollwöck, "Matrix product state algorithms: DMRG, TEBD and relatives," in *Strongly Correlated Systems*. Springer, 2013, pp. 67–98.

[213] C. J. Hillar and L.-H. Lim, "Most tensor problems are NP-hard," *Journal of the ACM*, vol. 60, no. 6, pp. 1–39, 2013.

[214] L. De Lathauwer, B. De Moor, and J. Vandewalle, "A multilinear singular value decomposition," *SIAM Journal on Matrix Analysis and Applications*, vol. 21, no. 4, pp. 1253–1278, 2000.

[215] U. Schollwöck, "The density-matrix renormalization group in the age of matrix product states," *Annals of Physics*, vol. 326, no. 1, pp. 96–192, 2011.

[216] F. Ju, Y. Sun, J. Gao, M. Antolovich, J. Dong, and B. Yin, "Tensorizing restricted Boltzmann machine," *ACM Transactions on Knowledge Discovery from Data*, vol. 13, no. 3, pp. 1–16, 2019.

[217] B. Pirvu, V. Murg, J. I. Cirac, and F. Verstraete, "Matrix product operator representations," *New Journal of Physics*, vol. 12, no. 2, p. 025012, 2010.

[218] A.-H. Phan, K. Sobolev, D. Ermilov, I. Vorona, N. Kozyrskiy, P. Tichavsky, and A. Cichocki, "How to train unstable looped tensor network," *arXiv preprint arXiv:2203.02617*, 2022.

[219] F. Sedighin and A. Cichocki, "Image completion in embedded space using multistage tensor ring decomposition," *Frontiers in Artificial Intelligence*, vol. 4, p. 687176, 2021.

[220] Y. Qiu, G. Zhou, C. Li, D. Mandic, and Q. Zhao, "Tensor ring rank determination using odd-dimensional unfolding," *Neural Networks*, p. 106947, 2024.

[221] F. Sedighin, A. Cichocki, T. Yokota, and Q. Shi, "Matrix and tensor completion in multiway delay embedded space using tensor train, with application to signal reconstruction," *IEEE Signal Processing Letters*, vol. 27, pp. 810–814, 2020.

[222] H. Huang, Y. Liu, and C. Zhu, "Low-rank tensor grid for image completion," *arXiv preprint arXiv:1903.04735*, 2019.

[223] D. Lahat, T. Adali, and C. Jutten, "Multimodal data fusion: an overview of methods, challenges, and prospects," *Proceedings of the IEEE*, vol. 103, no. 9, pp. 1449–1477, 2015.

[224] L. Morency, R. Mihalcea, and P. Doshi, "Towards multimodal sentiment analysis: harvesting opinions from the web," in *ICMI*, 2011.

[225] H. Wang, A. Meghawat, L. Morency, and E. P. Xing, "Select-additive learning: Improving cross-individual generalization in multimodal sentiment analysis," *CoRR*, vol. abs/1609.05244, 2016.

[226] G. Zhou, Q. Zhao, Y. Zhang, T. Adalı, S. Xie, and A. Cichocki, "Linked component analysis from matrices to high-order tensors: Applications to biomedical data," *Proceedings of the IEEE*, vol. 104, no. 2, pp. 310–331, 2016.

[227] J. Xu, J. Zhou, P.-N. Tan, X. Liu, and L. Luo, "Spatio-temporal multi-task learning via tensor decomposition," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 6, pp. 2764–2775, 2019.

[228] D. P. DiVincenzo, "Quantum computation," *Science*, vol. 270, no. 5234, pp. 255–261, 1995.

[229] M. Benedetti, E. Lloyd, S. Sack, and M. Fiorentini, "Parameterized quantum circuits as machine learning models," *Quantum Science and Technology*, vol. 4, no. 4, p. 043001, 2019.

[230] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *ECCV*, 2014.

[231] P. Molchanov, A. Mallya, S. Tyree, I. Frosio, and J. Kautz, "Importance estimation for neural network pruning," in *CVPR*, 2019.

[232] J. Yang, X. Shen, J. Xing, X. Tian, H. Li, B. Deng, J. Huang, and X. Hua, "Quantization networks," in *CVPR*, 2019.

[233] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, "Tinybert: Distilling BERT for natural language understanding," in *EMNLP*, 2020.

[234] A. Novikov, D. Podoprikhin, A. Osokin, and D. P. Vetrov, "Tensorizing neural networks," in *NeurIPS*, 2015.

[235] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[236] L. De Lathauwer, "Decompositions of a higher-order tensor in block terms—part II: Definitions and uniqueness," *SIAM Journal on Matrix Analysis and Applications*, vol. 30, no. 3, pp. 1033–1066, 2008.

[237] C. Lubich, T. Rohwedder, R. Schneider, and B. Vandereycken, "Dynamical approximation by hierarchical Tucker and tensor-train tensors," *SIAM Journal on Matrix Analysis and Applications*, vol. 34, no. 2, pp. 470–494, 2013.

[238] X. Ma, P. Zhang, S. Zhang, N. Duan, Y. Hou, M. Zhou, and D. Song, "A tensorized transformer for language modeling," *NeurIPS*, 2019.

[239] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020.

[240] S. Cheng, L. Wang, and P. Zhang, "Supervised learning with projected entangled pair states," *Physical Review B*, vol. 103, no. 12, p. 125117, 2021.

[241] N. Cohen and A. Shashua, "Convolutional rectifier networks as generalized tensor decompositions," in *ICML*, 2016.

[242] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *CVPR*, 2016.

[243] C. Li and Z. Sun, "Evolutionary topology search for tensor network decomposition," in *ICML*, 2020.

[244] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *AISTATS*, 2010.

[245] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on t classification," in *ICCV*, 2015.

[246] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *J. Mach. Learn. Res.*, vol. 20, no. 1, pp. 1997–2017, 2019.

[247] S. Nakajima, R. Tomioka, M. Sugiyama, and S. D. Babacan, "Perfect dimensionality recovery by variational Bayesian PCA," in *NeurIPS*, 2012.

[248] V. Pereyra and G. Scherer, "Efficient computer manipulation of tensor products with applications to multidimensional approximation," *Mathematics of Computation*, vol. 27, no. 123, pp. 595–605, 1973.

[249] S. van der Walt, S. C. Colbert, and G. Varoquaux, "The NumPy array: A structure for efficient numerical computation," *Comput. Sci. Eng.*, vol. 13, no. 2, pp. 22–30, 2011.

[250] M. Abadi, P. Barham, and et al., "Tensorflow: A system for large-scale machine learning," in *USENIX*, K. Keeton and T. Roscoe, Eds., 2016.

[251] T. Chen, M. Li, and et al., "Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems," *arXiv preprint arXiv:1512.01274, 2015*, 2015.

[252] X. Liu, "TenDeC++: Tensor decomposition library in c++," 2020. [Online]. Available: https://github.com/osmint/TenDeC

[253] X. Chen, Z. Li, L. He, and X. Liu, "Tensor4ml: Tensor decomposition for machine learning," https://github.com/xinychen/Tensor4ML, 2024, accessed: 2025-01-12.

[254] Y.-C. Hsu, T. Hua, S. Chang, Q. Lou, Y. Shen, and H. Jin, "Language model compression with weighted low-rank factorization," *arXiv preprint arXiv:2207.00112*, 2022.

[255] Z. Yuan, Y. Shang, Y. Song, Q. Wu, Y. Yan, and G. Sun, "Asvd: Activation-aware singular value decomposition for compressing large language models," *arXiv preprint arXiv:2312.05821*, 2023.

[256] M. Li, R. B. Basat, S. Vargaftik, C. Lao, K. Xu, M. Mitzenmacher, and M. Yu, "{THC}: Accelerating distributed deep learning using tensor homomorphic compression," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024, pp. 1191–1211.

[257] N. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles *et al.*, "Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.

[258] Z. Cai and J. Liu, "Approximating quantum many-body wave functions using artificial neural networks," *Physical Review B*, vol. 97, no. 3, p. 035116, 2018.

[259] K. Choo, G. Carleo, N. Regnault, and T. Neupert, "Symmetries and many-body excitations with neural-network quantum states," *Physical Review Letters*, vol. 121, no. 16, p. 167204, 2018.

[260] L. Cincio, J. Dziarmaga, and M. M. Rams, "Multiscale entanglement renormalization ansatz in two dimensions: Quantum ising model," *Physical Review Letters*, vol. 100, no. 24, p. 240603, 2008.

[261] K. Batselier, A. Cichocki, and N. Wong, "Meracle: constructive layer-wise conversion of a tensor train into a mera," *Communications on Applied Mathematics and Computation*, vol. 3, pp. 257–279, 2021.

[262] J. A. Reyes and E. M. Stoudenmire, "Multi-scale tensor network architecture for machine learning," *Machine Learning: Science and Technology*, vol. 2, no. 3, p. 035036, 2021.